

Technical Review Whitepaper

Architecture Review

Current Capabilities and Functional Gaps

A whole-system survey of the QtRocket simulation core, its three front ends, and its distance from a full-featured flight simulator, with OpenRocket as the comparison baseline.

Status	Review of the working tree at commit 254cd41 ("Major comment cleanup")
Scope	Every layer: <code>model/</code> , <code>sim/</code> , <code>utils/</code> , <code>gui/</code> , <code>cli/</code> , <code>visualizer/</code> , <code>build</code> , <code>tests</code> , <code>CI</code>
Target	QtRocket simulation core (C++23); baseline branch <code>development</code>
Audience	Maintainers and future contributors of the simulation core

Abstract

QtRocket is an open-source model-rocket simulator: a C++23 simulation core shared by a Qt6 Widgets GUI, a Qt-free headless REPL, and a standalone OpenGL design viewer. This whitepaper is a whole-system architecture review of the working tree at commit [254cd41](#) — what the program can do today, how its layers fit together, and where it stands relative to OpenRocket, used throughout as a comparison baseline for an independent project with similar goals, not as a specification to clone.

What exists is a genuinely finished, well-guarded 3-DOF point-mass simulator: a composite part tree whose placement is stored as physical intent and resolved by a single geometry resolver shared with the visualizer, time-aware composite mass/CG/inertia behind a mass-delta cache, motor ingest from RockSim files, RASP files, and a live thrustcurve.org client, a validated seven-layer US Standard Atmosphere 1976, pluggable gravity models, fixed-step RK4 and adaptive RKF45 integration, typed termination diagnostics around a hang-proof run loop, XML design persistence, and a 33-command scriptable REPL that exercises all of it. Around that core sits the review's central finding: a ring of machinery that is built, unit-tested, and consumed by nothing. The Barrowman center-of-pressure composition, the per-step inertia tensor (recorded every step, integrated by nothing), the wind model, the quaternion state slots, and the torque interface all exist and all have zero production consumers. Flight physics therefore remains thrust-along-world-vertical plus gravity plus a single constant- C_d drag term: no angle of attack, no restoring moment, no weathercocking, no recovery deployment — every flight ends ballistically. The GUI lags furthest behind the core: rocket design is possible only from the CLI, with the design-editor widgets empty shells at the reviewed commit and in-flight work on a separate branch.

The review was produced by eight independent subsystem deep-reads, each adversarially verified citation-by-citation against the source, plus live execution of the CLI binary; design documents under [docs/](#) were treated as claims to test, not facts to inherit. The result is a feature matrix against OpenRocket, a ranked inventory of functional deficiencies — flight-dynamics fidelity first, recovery second, component catalog and design UI close behind — and a suggested sequencing that consumes the already-built seams before building new ones.

The review in one sentence

QtRocket's architecture is ahead of its features: the discipline — fail-closed diagnostics, strict warnings, five-platform CI, verified seams — is that of a much larger simulator, and the shortest path to realistic flights is to *consume* the dormant machinery (Barrowman aerodynamics, torques, wind, recovery events), not to build new subsystems.

Contents

Abstract	ii
1 Introduction & scope	1
1.1 The tree under review	1
1.2 What the review found, in one paragraph	1
1.3 Method	2
1.4 Document roadmap	2
2 System overview & layering	2
2.1 Three executables, one core	2
2.2 The layering and the link graph	4
2.3 The controller and the Propagatable bridge	5
2.4 Build system in brief	5
3 The model layer: part tree, placement, and mass properties	6
3.1 Tree ownership, identity, and duplication	7
3.2 Frame convention and the StationLink placement model	7
3.3 Envelope-sweep diagnostics and the fail-closed gate	8
3.4 The two-pass composite walk and its caches	9
3.5 The concrete part catalog	10
3.6 RocketModel: tree root and the Propagatable bridge	12
4 The motor subsystem	12
4.1 Thrust: ThrustCurve	12
4.2 Mass: the 128-point impulse-proportional curve	13
4.3 Ignition and the two burnout clocks	14
4.4 Storage and search: MotorModelDatabase	15
4.5 Three ingest paths	15
4.6 The motor in flight	16
4.7 Bundled data	17
5 The simulation core	17
5.1 The injected ODE and what is actually integrated	18
5.2 The run loop and typed termination	19
5.3 The Integrator facade and the two solver backends	21
5.4 The Environment registry	22
5.5 The QtRocket controller	23
6 Aerodynamics and flight-physics fidelity	24
6.1 The consumed force path	24
6.2 The dormant Barrowman layer	26
6.3 Latent defects that must precede consumption	26
6.4 Degrees of freedom: the rotational half does not exist	27
6.5 The rest of the flight envelope	28

7	User interfaces: GUI, CLI, and visualizer	29
7.1	The Qt6 GUI: a motor-and-launch front end	29
7.2	The command-line REPL	31
7.3	The standalone visualizer	33
7.4	Work in flight: the RocketTreeViewImplementation branch	34
8	Persistence & interoperability	35
8.1	The .qrd design format	35
8.2	Round-trip fidelity	36
8.3	Loader error handling and versioning	37
8.4	The placement-invariance baseline	37
8.5	Motor database files (.qmd)	38
8.6	Import and export surface	38
9	Testing, coverage, and engineering infrastructure	39
9.1	Build system: presets, pinned dependencies, strict warnings	39
9.2	The test suite: six binaries, 239 cases	40
9.3	Coverage tooling	41
9.4	Continuous integration	42
9.5	The utils layer	42
9.6	Untested surfaces and dormant machinery	43
10	Functional comparison with OpenRocket	43
10.1	Four philosophical differences	44
10.2	Feature matrix	44
10.3	Deficiencies, ranked by flight-realism impact	47
10.4	Where QtRocket is ahead	48
11	Incomplete areas & future work	48
11.1	Incompleteness as a discipline	48
11.2	Consolidated inventory	49
11.3	Work in flight and branch archaeology	51
11.4	Deliberately deferred, not forgotten	52
11.5	Sequencing: what unlocks what	53
A	Reproducing this document (toolchain)	53
A.1	Directory layout and pipeline	54
A.2	How this review was produced	54

1 Introduction & scope

QtRocket is an open-source model-rocket simulator built as a shared C++23 simulation core with three front ends: a Qt6 Widgets GUI (`qtrocket`), a Qt-free headless REPL (`qtrocket-cli`), and a standalone OpenGL design viewer (`qtrocket-visualizer`). This document is a whole-system architecture review: a survey of what the program can actually do today, how its layers fit together, and — measured against OpenRocket as a reference point — what it cannot do yet. It is written for maintainers and future contributors who need an honest map of the territory before choosing what to build next.

Two framing points define the document’s stance. First, QtRocket is an *independent* project with design goals similar to OpenRocket’s, not a clone of it; OpenRocket is used throughout as a comparison baseline because it is the de facto standard for what a mature open-source model-rocket simulator offers, not because feature parity is the goal (Section 10). Second, the review reports the *code*, not the project’s aspirations: design documents under `docs/` were treated as claims to test against the source, never as facts to inherit, and where the two disagree this document says so.

1.1 The tree under review

Every statement in this document is pinned to one revision: commit `254cd41` (“Major comment cleanup”), reviewed with a clean working tree on 2026-07-06. That commit is the tip of the `development` line at the time of review and includes the completed Part Placement feature (the station-pair — `StationLink` — placement backbone specified in the companion whitepaper under `docs/PartPlacementDesignLatex/`). Two commits of in-flight GUI work exist beyond it on the `RocketTreeViewImplementation` branch — an initial part-tree widget and work-in-progress design save/load wiring — and are covered explicitly as future work (Sections 7 and 11) rather than folded silently into the review. Code citations take the form `model/parts/Part.h:42`; line numbers will drift as the tree moves and should be read as anchors into that revision, not absolutes.

1.2 What the review found, in one paragraph

The load-bearing finding is a pattern, not a single fact. QtRocket today has a genuinely finished, well-guarded 3-DOF point-mass flight simulator — composite part tree with time-aware mass, CG, and inertia; validated US Standard Atmosphere; adaptive RKF45 integration; typed termination diagnostics; motor ingest from three sources; design persistence; a scriptable REPL that exercises all of it — and, surrounding that core, a ring of machinery that is built, unit-tested, and consumed by nothing. The Barrowman center-of-pressure composition, the per-step composite inertia tensor (recorded, never integrated), the wind model, the quaternion slots in the state vector, and the `getTorques()` interface all exist and all have zero production consumers; the test suite says so itself (“nothing in production consumes this in P2,” `model/tests/AeroTests.cpp:18`). The project’s engineering discipline — fail-closed diagnostics, strict warnings, five-platform CI, pinned dependencies — is well ahead of its feature completeness. The architecture is not the obstacle to a realistic flight simulation; *consumption* of what the architecture already provides is.

How to read the gap findings

Throughout this document, “dormant” means built and tested but unreachable from any production code path; “partial” means reachable but with named gaps; “absent” means no trace in the tree (each absence claim was grounded in a failed search of the source, not an impression). The distinctions matter for planning: consuming a dormant seam is weeks of work where building

an absent subsystem is months.

1.3 Method

The review was produced by independent parallel deep-reads of eight subsystem areas — the part tree and placement system, the motor subsystem, the simulation core, aerodynamics and flight fidelity, the GUI and visualizer, the CLI and persistence, build and test infrastructure, and a dedicated cross-cutting incompleteness sweep — each grounded in a full read of its files and each followed by an adversarial verification pass that re-checked every file:line citation, every quoted snippet, and every implemented/dormant/absent status claim against the source. Two behaviors were confirmed by execution rather than reading: the CLI command inventory and an end-to-end flight through the built REPL binary. The OpenRocket baseline of [Section 10](#) was researched from primary sources and reconciled against the verified findings. [Section A](#) records the full method and the document’s own build pipeline.

1.4 Document roadmap

[Section 2](#) maps the system: the three executables, the strict layering, and the build that enforces it. [Sections 3 to 5](#) review the three pillars of the core — the part tree with its placement resolver and mass-property machinery, the motor subsystem from thrust curves to the three ingest paths, and the propagator with its integrators and environment models. [Section 6](#) is the keystone: what the force path actually computes, what sits dormant beside it, and what that means for flight fidelity. [Section 7](#) reviews the three user-facing surfaces and their very different levels of completeness; [Section 8](#) the design-file format and interoperability; [Section 9](#) the test suites, coverage tooling, and CI. [Section 10](#) sets the feature matrix against OpenRocket and ranks the functional deficiencies; [Section 11](#) consolidates every incomplete area found by the review into a severity-ordered inventory with a suggested sequencing. [Section A](#) closes with reproducibility: how this document builds and how the review was performed.

2 System overview & layering

QtRocket ships three executables built from one Qt-free simulation core. `qtrocket` is the Qt6 Widgets desktop application; `qtrocket-cli` is a headless, line-oriented REPL over the same simulation API; `qtrocket-visualizer` is a standalone OpenGL viewer for saved designs. All three link the same static library, `qtrocket_core`, which contains exactly one class — the `QtRocket` application controller — and pulls in the `model`, `sim`, and `utils` layer libraries beneath it (`CMakeLists.txt:224`). [Figure 1](#) shows the arrangement; the rest of this section walks it from the top down, then summarizes the build that enforces it.

2.1 Three executables, one core

qtrocket (GUI). The entry point instantiates the `Logger` at INFO `main.cpp:16`, obtains the controller singleton `main.cpp:20`, and blocks in `gui::run(qtrocket, argc, argv)` `main.cpp:25`. The executable is the only Qt-dependent front end in the main tree, linking `Qt6::Widgets`, `Qt6::PrintSupport`, and `qtrocket_core` `CMakeLists.txt:286`. One structural oddity is worth flagging at the map level: `gui::run` spawns a `std::thread` whose body constructs `QApplication` and runs the entire Qt event loop, then joins it `gui/GuiRunner.cpp:62` — an unusual inversion of Qt’s main-thread convention whose consequences [Section 7](#) examines.

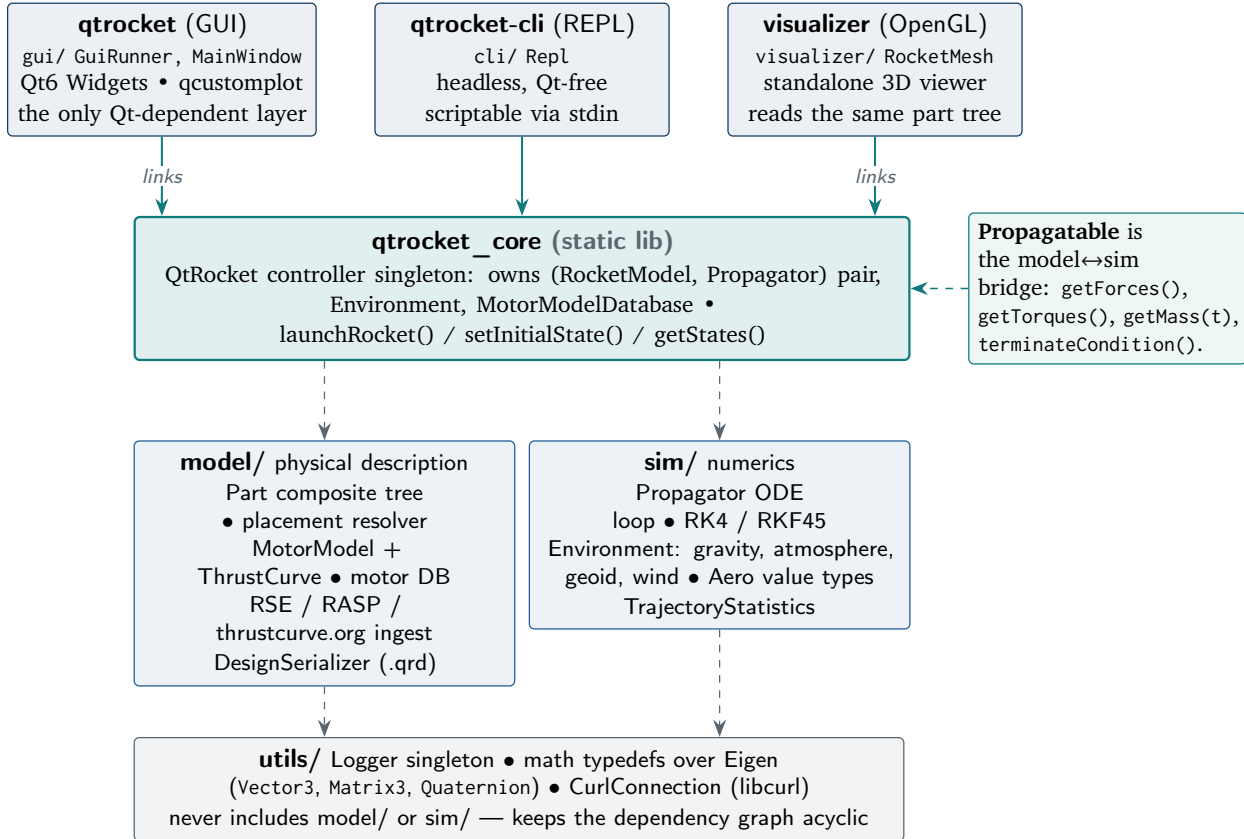


Figure 1. The shipped system: three executables over one Qt-free core. Every edge points strictly downward — front ends link qtrocket_core, the core links model/sim/utils, and utils links only third-party math and networking. The Propagatable interface (right) is the single seam through which the simulation loop sees the rocket model.

qtrocket-cli (headless REPL). The CLI entry point deliberately ignores `argc/argv cli/CliMain.cpp:13`, silences the logger to ERROR because `stdout` doubles as the machine-readable OK/ERR/CSV protocol `cli/CliMain.cpp:17`, prints a #-prefixed banner, and hands `std::cin/std::cout` to `cli::Repl cli/CliMain.cpp:25`. The Repl is compiled into its own cli static library so that the test binaries drive the identical object code users type into `CMakeLists.txt:234`; the executable links that library and nothing else `CMakeLists.txt:297`. [Section 7](#) reviews the command set itself.

qtrocket-visualizer (OpenGL viewer). A self-contained Qt application under `visualizer/` that finds its own `Qt6::OpenGLWidgets/OpenGL` components `visualizer/CMakeLists.txt:9` and, by stated design, never touches `gui/` or `cli/` code `visualizer/CMakeLists.txt:3`. Its main disables the RHI widget backing store `visualizer/main.cpp:22` and deliberately requests no specific GL version (a documented EGL/Wayland-plus-NVIDIA workaround, `visualizer/main.cpp:10`), then loads an optional `.qrd` design path from `argv[1]` `visualizer/main.cpp:35` through the shared `model::DesignSerializer` `visualizer/VisualizerWindow.cpp:184`.

The visualizer’s load path illustrates the document’s recurring motif early: the shared design serializer is consumed by the CLI and the visualizer, but at this commit the GUI — the front end a desktop user would expect to open files from — has no design load path at all; GUI save/load wiring exists only as work in progress on the `RocketTreeViewImplementation` branch ([Sections 7 and 8](#)). The seam was built and tested before its most obvious consumer arrived.

2.2 The layering and the link graph

The layering is not a documentation convention; it is the link graph. `utils` sits at the bottom, linking only `libcurl` and `eigen` `utils/CMakeLists.txt:14`. `sim` links only `utils` `sim/CMakeLists.txt:28`. `model` links `utils`, `sim`, `Boost::property_tree`, and `jsoncpp_static` `model/CMakeLists.txt:44` — so `model` and `sim` are layered rather than peers: the model depends on the simulation layer’s value types (`sim::Aero`, `StateData`), never the reverse `model/CMakeLists.txt:41`. Above both sits `qtrocket_core` ([Listing 1](#)), and above that the three executables plus the test binaries. Include resolution everywhere uses a single global `include_directories(${PROJECT_SOURCE_DIR})` `CMakeLists.txt:147`, which is what makes the `model/...`, `sim/...`, and `utils/...` path prefixes resolve uniformly across the tree.

```
# Application controller shared by the GUI, the CLI, and the
# integration tests.
add_library(qtrocket_core STATIC
    QtRocket.cpp
    QtRocket.h)
qtrocket_enable_coverage(qtrocket_core)

target_link_libraries(qtrocket_core PUBLIC
    model
    sim
    utils)
```

Listing 1. The keystone of the link graph `CMakeLists.txt:222`: the controller compiles once into a static library, and every executable and integration-test binary links it rather than recompiling `QtRocket.cpp`.

The layering invariant, stated where it binds

“Bottom layer: utils must not depend on model/ or sim/” `utils/CMakeLists.txt:13`. The comment sits directly above the `target_link_libraries` call it constrains, and the review confirmed it holds: no header under `utils/` includes anything from `model/` or `sim/`. This single rule is what keeps the dependency graph acyclic — every layer above `utils` may share its math typedefs and logging without creating a back-edge.

2.3 The controller and the Propagatable bridge

`QtRocket` is the application controller: a process-wide singleton `QtRocket.cpp:19` that owns the (`RocketModel`, `Propagator`) pair, the `Environment`, and the `MotorModelDatabase`. Installing a rocket constructs its propagator in the same breath, with the shared environment injected `QtRocket.h:41` — the model and its integrator loop are never allowed to drift apart. The API surface the front ends drive is deliberately small: accessors for the environment, time step, integrator selection, rocket, and motor database `QtRocket.h:34`; `launchRocket()` `QtRocket.h:43`; the recorded state history via `getStates()` `QtRocket.h:48`; whole-flight summary statistics `QtRocket.h:55`; a typed termination reason `QtRocket.h:62`; and `setInitialState()` `QtRocket.h:68`. A launch is four steps — clear the state history, reset propagator time to zero, ignite the motor, run the ODE loop to termination `QtRocket.cpp:54` — and everything else (staging launch parameters, choosing models, exporting results) lives in the front ends.

Racy double-checked initialization

`QtRocket::getInstance()` reads the static `initialized` flag without holding the mutex `QtRocket.cpp:21`; only `init()` re-checks under the lock `QtRocket.cpp:30`. Two threads calling `getInstance()` concurrently would race on that unsynchronized read — a data race in the C++ memory model. It is latent today because the two front ends that use it call it once, single-threaded, at startup (the visualizer builds its own `RocketModel` and never touches the controller); but the GUI’s spawned-thread event loop ([Section 7](#)) means the assumption is worth stating rather than trusting. The `Logger` singleton one layer down already uses the safe Meyers form `utils/Logger.cpp:19`.

The seam between the two halves of the core is the `Propagatable` interface `model/Propagatable.h:23`: the only view of the rocket that the simulation loop ever sees. `RocketModel` implements it; the `Propagator` consumes it. Its pure-virtual heart is four questions the integrator asks at every step ([Listing 2](#)), plus `terminateCondition()` `model/Propagatable.h:39`, which lets the model rather than the loop decide when a flight is over. Only the three linear degrees of freedom are integrated today, so the force half of the bridge does all the work; what the torque half is waiting for is reviewed with the aerodynamics in [Section 6](#), and the loop that drives the interface in [Section 5](#). The mass-property machinery behind `getMass(t)` — the composite part tree — is the subject of [Section 3](#).

2.4 Build system in brief

The build is a single CMake project — `cmake_minimum_required(VERSION 3.25)` `CMakeLists.txt:1`, `VERSION 0.1` `CMakeLists.txt:3` — pinned to C++23 `CMakeLists.txt:5` with `compile_commands.json` exported for tooling `CMakeLists.txt:7`. Generator, compiler, and build type are pinned by `CMakePresets.json`: Ninja everywhere `CMakePresets.json:7`, with eight configure presets — `debug/release` for the default compiler, Clang, and GCC, a release-only MSVC preset

```

virtual Vector3 getForces(double t, const Vector3& position, const Vector3& velocity, sim::Environment&
    ↪ environment) = 0;
virtual Vector3 getTorques(double t) = 0;

virtual double getMass(double t) = 0;
virtual Matrix3 getCompositeInertiaTensor(double t) = 0;

```

Listing 2. The model↔sim bridge `model/Propagatable.h:29`: the integrator needs forces, torques, and time-varying mass properties, and nothing else about the rocket.

`CMakePresets.json:70`, and `coverage-clang`, which redirects to a separate `build-coverage/` tree `CMakePresets.json:35` so instrumentation never disturbs the normal build. Matching build and test presets exist for each, and the same presets drive both developer workflows and CI.

Dependency	Pinned tag	Build shape and role	
GoogleTest	v1.17.0	test framework for all six test binaries	<code>CMakeLists.txt:17</code>
jsoncpp	1.9.7	static-only, tests/examples off; thrustcurve.org JSON	<code>CMakeLists.txt:32</code>
curl	curl-8_20_0	static, HTTP-only, optional backends forced off	<code>CMakeLists.txt:44</code>
Eigen	5.0.1	vectors, matrices, quaternions beneath <code>utils</code>	<code>CMakeLists.txt:72</code>
Boost	boost-1.91.0-1	restricted to <code>property_tree</code> ; XML persistence	<code>CMakeLists.txt:79</code>
Qt6	system	Widgets/PrintSupport; visualizer adds OpenGL modules	<code>CMakeLists.txt:140</code>

Table 1. Every non-Qt dependency is vendored via `FetchContent` at an exact tag; Qt6 is the sole system dependency. The first configure therefore downloads and compiles the world — slow once, hermetic thereafter.

Dependency policy is strict vendoring (Table 1). The curl configuration is the most deliberate: built static and HTTP-only `CMakeLists.txt:47` with every optional third-party backend explicitly forced off, because auto-detected Homebrew libraries on macOS CI runners had previously broken the final link — the hermeticity rationale is recorded in the tree `CMakeLists.txt:51`. Qt is resolved with a hard `find_package(Qt6 REQUIRED ...)` `CMakeLists.txt:140`; the finer hygiene findings in that block are examined in Section 9.

Warning hygiene is tree-wide and strict: `-Wall -Wextra -Wpedantic -Werror` on GCC and Clang `CMakeLists.txt:97`, `/W4 /WX` on MSVC `CMakeLists.txt:88`, applied after the `FetchContent` blocks so vendored code is exempt. The test suites themselves (six GoogleTest binaries), the five-platform CI matrix that runs them, the `llvm-cov` coverage target, and the finer warning-hygiene findings (a stale sanitizer claim, the vendored `gui/qcustomplot.cpp` exemption) are reviewed in depth in Section 9.

Finally, what the build does *not* yet do: distribution. QtRocket today builds well everywhere and installs properly nowhere — the install rule, bundle metadata, and translation rules are template stubs, examined with the rest of the packaging story in Section 9; Section 11 places that gap among the others.

3 The model layer: part tree, placement, and mass properties

The model layer describes a rocket as a composite tree of `model::part::Part` nodes, each of which is simultaneously one component and the root of a sub-tree whose mass, center of mass, and inertia tensor it aggregates on demand (`model/parts/Part.h:25`). Three mechanisms define the layer: an ownership tree with move-only attachment, stable ids, and polymorphic deep-copy; an intent-based placement model whose only stored spatial relationship is a `StationLink`, every absolute pose being

derived by a single resolver; and a two-pass, time-aware mass-properties walk cached behind a mass-delta gate. The layer also exhibits, in concentrated form, this review’s recurring motif: several of its seams — the reserved placement quaternion, the per-seam radial check, the live-length re-resolution machinery, the Barrowman composition — are finished and tested but have no production consumer yet.

3.1 Tree ownership, identity, and duplication

`Part` is an abstract base — `typeName()` and `cloneShallow()` are pure (`model/parts/Part.h:181`, `model/parts/Part.h:217`) — owning its children as a vector of `std::pair<std::shared_ptr<Part>, StationLink>` (`model/parts/Part.h:288`), with a non-owning `Part*` parent used solely to propagate staleness upward (`model/parts/Part.h:222`). At the value level a part is neither copyable nor movable — assignment is deleted and the copy constructor protected, so a subclass cannot be sliced and “a part lives at one place in one tree” (`model/parts/Part.h:66`). Attachment transfers ownership by move: `addChildPart` rejects a null child, an already-parented child, and any cycle-forming attachment as logged no-ops (`model/parts/Part.cpp:73`), then re-parents, emplaces, and dirties both the composite and placement caches up the ancestor chain (`model/parts/Part.cpp:97`). `removeChildById` moves the owning pointer out, clears the detached node’s parent, dirties the ex-parent chain, and returns the intact sub-tree (`model/parts/Part.cpp:277`).

Identity is separated from naming: each part gets a process-unique `Id` from a relaxed atomic counter starting at 1, with 0 reserved as none/invalid (`model/parts/Part.cpp:28`); ids survive adoption, are refreshed on copy, and are deliberately not serialized (`model/parts/Part.h:171`). Duplication has one path: `clone()` calls the pure `cloneShallow()` (implemented per type via the protected copy constructor, `model/parts/BodyTube.h:63`) to reproduce the node’s dynamic type, then recursively clones children and copies each `StationLink` verbatim (`model/parts/Part.cpp:106`). One consequence deserves stating plainly: apart from `setMass/setI` (`model/parts/Part.h:74`), a part is immutable after construction — no concrete type exposes a geometry setter (`model/parts/BodyTube.h:69`), so changing a dimension means building a replacement part and losing the id.

3.2 Frame convention and the `StationLink` placement model

Every part lives in its own $+z$ =forward frame, occupying $z \in [-L, 0]$ with the fore plane at the local origin (`model/parts/Part.h:130`). Geometry is exposed through closed-form silhouette virtuals `radiusOuterAt/radiusInnerAt` (`model/parts/Part.h:138`), an `axialLength()` envelope span defaulting to `getLength()` (`model/parts/Part.h:146`), an `isSolid()` heuristic (`model/parts/Part.h:151`), and `stationAt(station01)`, which clamps to $[0, 1]$ and maps the fraction to $z = (s - 1)L$ (`model/parts/Part.cpp:305`). `innerCapacityAt` encodes the solid-host rule the diagnostics rely on: a bored part is bounded by its inner wall, a solid part by its outer skin (`model/parts/Part.cpp:314`).

Placement is expressed as intent, not coordinates. A `StationLink` carries a fractional station on the parent (default 0, aft plane), one on the child (default 1, fore plane), a gap in metres whose sign convention is selected by a `SeatKind`, and a quaternion `childRot` “reserved for 6-DOF; identity in 3-DOF” (`model/parts/Placement.h:49`). `SeatKind` is closed at three values — `Abut` (rim-to-rim, gap a forward standoff), `NestInBore` (child OD inside parent ID, gap an insertion depth moving the child aft), and `OnSurface` (child on the parent’s outer wall) — since a rigid axisymmetric stack admits only three radial relationships (`model/parts/Placement.h:32`). Sugar verbs `abut`, `nestInBore`, and `seatOnWall` build links by value (`model/parts/Placement.h:127`); the default link means “child fore plane abuts parent aft plane” (`model/parts/Placement.h:46`). A single resolver converts intent to geometry (`model/parts/Placement.cpp:20`):

```
const double signedGap = (link.seat == SeatKind::NestInBore) ? -link.gap : link.gap;

// Line the child's station up with the parent's, then displace by the gap. The pose locates the
// child's fore plane, which sits c.z from its chosen station -- hence the '- c.z'.
const double childOriginZ = p.z + signedGap - c.z;
```

Both stations are read at live lengths and composition is coaxial in 3-DOF (`model/parts/Placement.cpp:34`); `Pose` pairs a fore-plane origin with a quaternion whose `compose()` is the general rigid-transform law, degenerating to vector addition under identity (`model/parts/Placement.h:74`); `resolvePlacements` is a depth-first walk yielding one pose per part in deterministic attachment order (`model/parts/Placement.cpp:38`).

Placement is derived, never stored

Absolute pose exists only as resolver output — “the single authority for absolute placement: intent (`StationLink`) in, geometry (`Pose`) out. Both the simulator and the visualizer consume its output, so the two cannot disagree” (`model/parts/Placement.h:148`). The serializer honors the rule: a `.qrd` file stores only `<link>` intent and re-resolves on load (`model/DesignSerializer.cpp:93`; [Section 8](#)).

Two caveats temper the picture. The stations are documented against *live* length — “editing a part’s length moves the seam” (`model/parts/Placement.h:42`), echoed by the placement-gate comment’s “or a length changed” (`model/parts/Part.cpp:162`) — yet no geometry mutation path exists, so the live-length machinery is future-proofing with no current trigger. And the authoring surface is far narrower than the model: the REPL’s `addpart` attaches everything with `abut(offset.z())` (`cli/Repl.cpp:872`), leaving `NestInBore`, `OnSurface`, and non-default stations reachable only from a design file or C++ (`model/DesignSerializer.cpp:186`; [Section 7](#)).

3.3 Envelope-sweep diagnostics and the fail-closed gate

Of the two diagnostic layers shipped, exactly one is wired in. Layer 2, `sweepOverlaps`, is the enforced gate: it builds a world axial interval per part from `axialLength()` (`model/parts/Placement.cpp:112`), excludes an offender’s legitimate neighbours — itself, its descendants, its direct seat parent (`model/parts/Placement.cpp:124`) — sorts intervals by aft station (`model/parts/Placement.cpp:151`), samples each offender’s outer radius at feature breakpoints (its own endpoints plus every other endpoint within its span, `model/parts/Placement.cpp:160`), tests it against the smallest-id covering host’s capacity under the solid-host rule (`model/parts/Placement.cpp:185`), and keeps the worst violation per (offender, host) pair (`model/parts/Placement.cpp:201`). A hollow-host guard suppresses the false positive of an offender sitting on or outside the host skin, e.g. a fin set’s body disc over a co-radial aft coupler (`model/parts/Placement.cpp:193`); diagnostics name parts by stable id, never by name or pointer (`model/parts/Placement.h:99`). Resolve and verdict are cached per structural change — mass edits never set `placementDirty` (`model/parts/Part.cpp:160`) — and consumed fail-closed: `computeCompositeAt` refuses a failed solve with a `std::runtime_error` carrying the first located diagnostic, on the stated principle that “a self-intersecting design is a hard, located failure, never a silently-wrong mass/inertia” (`model/parts/Part.cpp:182`), and the visualizer reads the same cached verdict to flag the offender (`visualizer/RocketMesh.cpp:440`). The sweep runs once per edit, so the per-step cost is a flag read (`model/tests/DiagnosticsGateTests.cpp:66`).

Three things bound what the sweep checks. The first is a correctness hole:

The monotonicity premise fails for the sphere

The breakpoint scheme is justified in-source by “profiles are closed-form and monotone between breakpoints, so this is exact, not sampled” (`model/parts/Placement.cpp:161`). That premise is false for `HollowSphere`, whose silhouette $\sqrt{r_o^2 - (z + r_o)^2}$ is zero at both poles — its own interval endpoints — and maximal at the mid-span equator (`model/parts/HollowSphere.cpp:53`). A sphere nested inside a tube narrower than its diameter evaluates radius ≈ 0 at every breakpoint (unless another part’s endpoint happens to land near the equator) and passes the sweep undetected. No test exercises a sphere in the sweep: the fixtures are tubes, cones, and fins (`model/tests/ResolverSweepTests.cpp:188`). **FAIL**

The second is a scoping rule: at each breakpoint the sweep tests capacity against a *single* host — the smallest id among covering, non-excluded intervals (`model/parts/Placement.cpp:185`) — so a violation against a larger-id covering host goes unevaluated at that station. The tie-break is pinned by test as intended (`model/tests/ResolverSweepTests.cpp:325`), but it makes the sweep a per-breakpoint single-host check, not an all-hosts check. The third is the dormant Layer 1: `radialSeamCheck` validates one seam’s mated radii per `SeatKind` (`model/parts/Placement.cpp:56`) and is implemented and unit-tested, but nothing in production calls it — not `ensurePlacementCache`, not `addChildPart`, not the serializer or CLI (`model/parts/Part.cpp:170`; call sites only in `model/tests/ResolverSweepTests.cpp:194`). Its `zWorld` is even left parent-local behind a comment promising “the root-frame z is added once a pose is known” (`model/parts/Placement.cpp:87`) — a promise no caller exists to fulfil. This is the built-but-unconsumed motif in miniature.

3.4 The two-pass composite walk and its caches

The tensor convention drives everything else: the bare tensor (`getI/setI`) is *per-unit-mass* (geometric, m^2) for this part alone, while `getCompositeI` is the full mass-weighted tensor (kg m^2) of the sub-tree about the composite CM (`model/parts/Part.h:30`); the constructor seeds the composite as $\mathbf{m} * \mathbf{I}$ (`model/parts/Part.cpp:41`). All closed-form tensors in `model::InertiaTensors` — spheres, tube, solid cone, conical shell, and the trapezoidal fin set with its baked-in N -fin rotate-and-sum (`model/InertiaTensors.h`, `model/InertiaTensors.cpp:11`) — follow the per-unit-mass convention and are pinned against numeric-integration oracles (`model/tests/InertiaTensorsTests.cpp:186`; Section 9).

`computeCompositeAt(t)` re-weights the cached geometry by `getMass(t)`. Pass 1 forms the mass-weighted composite CM, locating each part at its resolved fore-plane origin plus the local CM station $-L/2 + \text{getCenterMassOffset}().z()$ (`model/parts/Part.cpp:199`), with a guard that leaves a fully massless sub-tree’s CM at the origin rather than dividing by zero (`model/parts/Part.cpp:210`). Pass 2 shifts each part’s mass-weighted tensor to that CM with the parallel-axis displacement tensor $f(d) = (d \cdot d) I_3 - d d^T$ (`model/parts/Part.cpp:19`), accumulating $c.\text{mass} * c.\text{part} \rightarrow \text{getI}() + c.\text{mass} * \text{parallelAxisTerm}(d)$ (`model/parts/Part.cpp:221`). What pass 2 deliberately omits is the $R I R^T$ rotation of each child tensor — identity in 3-DOF, skipped “to stay bit-stable” with an explicit `TODO(6-DOF)` (`model/parts/Part.cpp:217`); the class doc names the inertia shift as “the one piece still assuming identity” (`model/parts/Part.h:38`). The omission already shapes the catalog: an azimuthally-arrayed component cannot be N rotated children, which is precisely why `FinSet` bakes its N -fin sum into one analytic leaf (`model/parts/FinSet.h:25`).

The result sits behind two gates: the structural flag (set by `setMass/setI` and the tree mutators, propagated up the parent chain, `model/parts/Part.h:239`), and the *mass-delta* gate (`model/parts/Part.cpp:136`):

```
const double mNow = getCompositeMass(t);
```



```

if(needsRecomputing || mNow != builtAtCompositeMass)
{
    const CompositeProperties c = computeCompositeAt(t);
    compositeMass      = c.mass;
    compositeCm        = c.cm;
    compositeInertiaTensor = c.inertia;
    builtAtCompositeMass = mNow;
    needsRecomputing     = false;
}

```

The exact `!=` against a NaN-seeded sentinel (`model/parts/Part.h:248`) is deliberate: during a burn the motor mass differs every step, so CM and tensor recompute every step; after burnout `getMass(t)` repeats bit-for-bit and the cache freezes, immune to the integrator's repeated and rejected stage-time queries. `getCompositeMass(t)` itself stays an ungated cheap live sum — the ODE divisor and the gate key, doing no tensor or placement work (`model/parts/Part.cpp:118`) and deliberately not diagnostics-gated (`model/tests/DiagnosticsGateTests.cpp:81`).

The gate keys on total mass alone

The mass-delta gate compares only the composite total (`model/parts/Part.cpp:137`). A second time-varying-mass part whose burn kept the total constant while shifting the CM would be served a stale CM and tensor. Under the single-motor assumption (Section 3.6) this cannot arise, but the gate's soundness is coupled to that assumption, and no test covers two simultaneous burners.

One adjacent capability merits a single paragraph here. `getCompositeAero(refArea)` folds each part's Barrowman contribution additively over the resolved placement, re-expressing every x_{cp} onto the sub-tree-root (tip) datum — the same datum as `getCompositeCm`, so $cp - cg$ is directly the static margin (`model/parts/Part.cpp:231`) — with the reference area taken as the widest frontal disc, not a sum and not inflated by fins (`model/parts/Part.h:166`). The machinery is implemented and well tested at the model layer, but its only callers are tests; the flight path does not consume it (Section 6).

3.5 The concrete part catalog

Five concrete types exist (`model/parts/Parts.h:4`); Table 2 summarizes them. Construction is mediated by `PartFactory`: one type-agnostic optional-field struct `PartParams` is shared by the CLI parser, `makePart()`, and `params()` reflection so the field vocabulary cannot drift (`model/parts/PartFactory.h:27`); `makePart` dispatches on the exact `typeName()` string and throws on missing fields or unknown types (`model/parts/PartFactory.cpp:27`).

Measured against what a real rocket needs, the catalog is sparse. Missing outright are transitions and boattails, curved nose profiles, inner and motor-mount tubes, tube couplers as a first-class type (a coupler today is a nested `BodyTube`), centering rings, bulkheads, launch lugs and rail buttons, recovery devices (parachutes, streamers, shock cords), discrete mass objects, and pods or strap-on boosters (`model/parts/PartFactory.cpp:75`). **Absent** One convention is worth flagging as convention rather than constraint: every committed `.qrd` fixture mounts its `FinSet` with `seat="OnSurface"` on a `BodyTube` (e.g. `tests/data/designs/large38_basic.qrd:13`), but nothing enforces this — a REPL-authored fin set gets an `Abut` link. The quantitative OpenRocket comparison is consolidated in Section 10; the ranked gap list, in Section 11.

Part	Geometry and mass model	Distinctives and limits
BodyTube	Uniform-density hollow cylinder, $m = \rho \pi (r_o^2 - r_i^2) L$ (model/parts/BodyTube.cpp:28); solid rod when $r_i = 0$ (model/parts/BodyTube.h:55).	Ctor validates $0 \leq r_i < r_o$, $L > 0$, $\rho > 0$ (model/parts/BodyTube.cpp:22); drag contribution left 0, skin friction named “its eventual contribution” (model/parts/BodyTube.cpp:38).
ConicalNoseCone	Solid cone or thin lateral shell (model/parts/ConicalNoseCone.h:21); CM at $L/4$ (solid) or $L/3$ (shell) from the base (model/parts/ConicalNoseCone.cpp:49).	“A straight right circular cone” is the only profile (model/parts/ConicalNoseCone.h:22) — no ogive, parabolic, elliptical, power-series, or Haack shapes.
FinSet	$N \geq 3$ identical trapezoidal flat-plate fins as <i>one</i> analytic leaf (model/parts/FinSet.h:22); axial extent equals root chord (model/parts/FinSet.h:61).	$N < 3$ warns and reuses the $N \geq 3$ approximation rather than throwing (model/parts/FinSet.cpp:36); reference area is the body disc πr_b^2 , not the fin extent (model/parts/FinSet.h:71); only trapezoidal planforms.
HollowSphere	Thick-walled shell centered at $z = -r_o$, spanning $[-2r_o, 0]$ (model/parts/HollowSphere.h:51).	Overrides <code>isSolid()</code> — the bore vanishes at the poles (model/parts/HollowSphere.h:59); doubles as the boot placeholder body (model/RocketModel.cpp:27).
Motor	Zero-length leaf wrapping a <code>MotorModel</code> by value; <code>getMass(t)</code> returns the burn-time mass (model/parts/Motor.h:40); tensor is a constant solid cylinder, or zero when geometry is unknown (model/parts/Motor.cpp:25).	The one override that makes the tree’s mass, CG, and inertia time-aware (model/parts/Motor.h:18). No geometric envelope, so motor fit is never checked by the sweep (model/RocketModel.cpp:118); not factory-constructible (model/parts/PartFactory.cpp:73).

Table 2. The concrete part catalog at commit 254cd41.

3.6 RocketModel: tree root and the Propagatable bridge

`RocketModel` holds the tree root and implements `Propagatable`, the model↔sim bridge (`model/Propagatable.h:23`, `model/RocketModel.h:31`). On the mass side — this section’s half of the bridge — it is a thin, honest delegation: `getMass(t)` returns `topPart->getCompositeMass(t)` with the motor included as a child part, `setMass` writes only the top part’s dry mass (`model/RocketModel.cpp:36`, `model/RocketModel.h:87`), `getCompositeInertiaTensor` delegates to `getCompositeI` (`model/RocketModel.cpp:43`), and `writeMassProperties` snapshots the mutually consistent gated mass/CG/inertia triple into `StateData` per recorded step (`model/RocketModel.cpp:53`; called at `sim/Propagator.cpp:132`). `terminateCondition` ends the flight when descending below the launch site, $z < 0$ and $v_z < 0$ (`model/RocketModel.cpp:62`). The force side — world- z thrust “assumed through the CM” (`model/RocketModel.cpp:70`), a flat $C_d A$ drag term, `getTorques` returning zero (`model/RocketModel.cpp:94`) — belongs to [Section 6](#); the standing consequence here is that the composite $I(t)$ and $CG(t)$ the tree so carefully maintains have, today, exactly one consumer: the per-step state snapshot.

The part-tree facade keeps a borrowed `part::Motor*` handle consistent across edits: `setRoot`, `addPart`, and `removePart` all re-resolve it by a depth-first search for the *first* `Motor` node — an explicit single-motor assumption (`model/RocketModel.cpp:13`); a second motor in the tree would be silently ignored by thrust. `setMotorModel` creates the `Motor` child on first call and swaps the wrapped model in place on later calls, so the borrowed pointer never dangles (`model/parts/Motor.cpp:16`); where that child seats — at the airframe CM with a hard-coded zero offset, one motor, no stages — is [Section 4](#)’s subject. Two smaller dormant seams round out the picture: `deriveReferenceAreaFromGeometry()` computes the Barrowman-convention reference area from the tree (`model/RocketModel.cpp:48`) but has no production caller — the drag model runs on a hard-coded 38 mm-tube default (`model/RocketModel.h:141`; [Section 6](#)) — and `Propagatable` still carries a default-constructed `sim::Aero` `aeroData` member that nothing reads (`model/Propagatable.h:61`, `sim/Aero.h:63`). Both fit the layer’s character: the seams toward 6-DOF and real aerodynamics are built and tested here first, and [Section 11](#) ranks what it will take to consume them.

4 The motor subsystem

The motor subsystem is `QtRocket`’s most complete vertical slice: three independent ingest paths (RockSim XML, RASP text, and the thrustcurve.org REST API) funnel into a single source-agnostic database, which feeds a thrust-and-mass model that plugs into the composite part tree so that the rocket’s mass, center of gravity, and inertia tensor track the burn ([Section 3](#)). It is also the clearest exhibit of the document’s recurring motif: the seams are built and tested ahead of their consumers. The remote-source interface has a fake injected through a deliberate test seam and the ignition API accepts an arbitrary start time, yet production code ignites exactly one motor at exactly $t = 0$; ejection delays are parsed by every loader, persisted, and consumed by nothing.

4.1 Thrust: ThrustCurve

`ThrustCurve` stores a vector of (time, thrust) samples in seconds and Newtons, the maximum sample time, and an ignition-time offset (`model/ThrustCurve.h:12`). `getThrust(t)` subtracts the ignition offset, returns 0 N outside $[0, t_{\max}]$ (`model/ThrustCurve.cpp:50`), returns exact sample values on a hit, and linearly interpolates inside an interval (`model/ThrustCurve.cpp:47`). A default or empty curve degenerates to a single (0, 0) point so the interval walk always has something to stand on (`model/ThrustCurve.cpp:17`).

One boundary case earns attention because it used to be a memory-safety bug. Many published curves begin at $t > 0$ — the first sample is the first *measured* point, not an origin. When the query time precedes that first sample, the interval start must be an implicit (0, 0) origin:

```
// Interval start is the previous sample -- unless t precedes the first sample (i == begin), where
// the curve has no (0,0) origin and we ramp from it. std::prev(begin()) here would read out of
// bounds.
double tStart = 0.0;
double thrustStart = 0.0;
if(i != thrustCurve.cbegin())
{
    tStart = std::prev(i)->first;
    thrustStart = std::prev(i)->second;
}
```

Listing 3. The implicit-origin ramp in `model/ThrustCurve.cpp:76`.

Why the origin ramp matters

Before this guard, the interval walk computed `std::prev(i)` with `i` at `begin()` — an out-of-bounds heap read whose value depended on allocation layout, so early-burn thrust, and therefore apogee, varied from run to run. The fix is pinned by a named regression test (`model/tests/ThrustCurveTests.cpp:46`), and the interpolation semantics — exact samples, midpoints, pre-ignition and post-burnout zeros — are pinned alongside it (`model/tests/ThrustCurveTests.cpp:28`).

OK

4.2 Mass: the 128-point impulse-proportional curve

`MotorModel` pairs a static `MetaData` block — manufacturer, burn time, impulse class, delays, propellant and total weight in kilograms (`model/MotorModel.h:285`) — with runtime state: `ignitionOccurred`, `burnOutOccurred`, `emptyMass`, `isp`, `ignitionTime`, and a precomputed `massCurve` (`model/MotorModel.h:337`). The data member is public with a standing TODO to make it private (`model/MotorModel.h:282`); it is mutated freely across the codebase in the meantime.

Motor mass over the burn is not taken from data files. Installing metadata triggers `computeMassCurve()` (`model/MotorModel.cpp:85`), which sets $m_{\text{empty}} = m_{\text{total}} - m_{\text{prop}}$, computes a specific impulse $I_{\text{sp}} = I_{\text{tot}} / (g_0 m_{\text{prop}})$ with $g_0 = 9.80665 \text{ m/s}^2$ (`utils/math/Constants.h:10`) — a value that is then never read anywhere in the tree — and forward-Euler-depletes the propellant in proportion to the fraction of total impulse delivered, i.e. a constant- I_{sp} assumption ([Listing 4](#)).

`getMass(simTime)` then has three phases: loaded weight before ignition, empty mass plus interpolated propellant during the burn, and empty (casing) mass after burnout (`model/MotorModel.cpp:28`). This is the same fallback a mature tool uses when a file carries no measured mass data — but `QtRocket` applies it *always*, discarding the per-sample mass and CG measurements the bundled `RockSim` file actually provides ([Section 4.5](#)).

Curve before metadata

Because `computeMassCurve()` integrates the thrust curve, the curve must be installed before the metadata. All three construction sites honor the ordering; it is stated in a comment at two of them — “Order matters” at `model/MotorModelDatabase.cpp:303` and an equivalent note at `model/ThrustCurveClient.cpp:328` — while `model/RSEDatabaseLoader.cpp:96` performs the same ordering

```

// Precompute the mass curve as a lookup table: keeps repeated getMass()/getThrust() calls within
// one time step consistent, and speeds up the sim. 128 sample points (RASP files cap at 32).
massCurve.reserve(128);
double timeStep = data.burnTime / 127.0;
double t = 0.0;
double propMass{data.propWeight};
for(std::size_t i = 0; i < 127; ++i)
{
    massCurve.push_back(std::make_pair(t + i*timeStep, propMass));
    propMass -= thrust.getThrust(t + i*timeStep) * timeStep * data.propWeight / data.totalImpulse;
}
massCurve.push_back(std::make_pair(data.burnTime, 0.0));

```

Listing 4. `computeMassCurve()` resamples the thrust integral into a 128-point propellant lookup table (`model/MotorModel.cpp:132`).

silently. Nothing enforces it: a caller that reverses the two calls gets a mass curve integrated over an empty curve, with no diagnostic.

The computation carries latent hazards, none reachable through current call paths. `computeMassCurve` divides by `propWeight`, `totalImpulse`, and `burnTime` without zero guards (`model/MotorModel.cpp:135`), so defaulted metadata yields `inf/NaN` table entries; `massCurve` is reserved and appended but never cleared (`model/MotorModel.cpp:134`), so a second `setMetaData` on the same instance — permitted by the public API (`model/MotorModel.cpp:85`), never done by any loader — would append a second ramp and break the monotonic interval walk; and `getMass` dereferences `massCurve.cbegin()` with no empty check (`model/MotorModel.cpp:40`), so a default-constructed motor that is started and queried mid-burn is undefined behavior.

4.3 Ignition and the two burnout clocks

The runtime ignition API is one line: `void startMotor(double startTime) { ignitionOccurred = true; ignitionTime = startTime; }` (`model/MotorModel.h:328`). The parameter suggests staged or delayed ignition; the plumbing does not deliver it. `getMass` honors the offset (it evaluates the curve at `simTime - ignitionTime`, `model/MotorModel.cpp:36`), but `getThrust` passes *absolute* simulation time to the curve (`model/MotorModel.cpp:82`), and the curve's own offset never moves: `ThrustCurve::setIgnitionTime` exists (`model/ThrustCurve.cpp:41`, with its `maxTime` adjustment commented out at line 44) and has zero call sites in the tree. `getThrust` also never consults `ignitionOccurred`, so an un-started motor still reports curve thrust. All of this is masked by a single fact: `RocketModel::launch()` is the only production ignition and always calls `startMotor(0.0)` (`model/RocketModel.cpp:108`). Any future `startMotor(t ≠ 0)` would burn propellant on one clock and produce thrust on another.

Mass and thrust burn out on different clocks

Even at $t = 0$ ignition the two sides use different burnout criteria: `getMass` gates the burn phase on the *metadata* field `data.burnTime` (`model/MotorModel.cpp:36`), while `getThrust` gates on the *curve's* last sample time (`model/MotorModel.cpp:72`). RASP-derived motors are immune because the loader defines `burnTime` as the last sample time, but RockSim files supply the two independently — the bundled data carries a `burn-time` attribute alongside its sample list (`data/Aerotech.rse:6`) — so a file whose attribute disagrees with its samples produces a window in which thrust is zero

while propellant is still depleting, or vice versa. No test covers the disagreement. **FAIL**

4.4 Storage and search: `MotorModelDatabase`

`MotorModelDatabase` is the single ingest and selection surface: the RSE and RASP loaders are owned and consumed inside it (`model/MotorModelDatabase.cpp:63`), the remote client sits behind it, and the GUI and CLI only ever call the database. Internally it is a `std::map<std::string, MotorModel>` keyed by motor common name (`model/MotorModelDatabase.h:129`). The key choice has consequences: same-name motors from different sources silently replace one another, logged only at debug level (`model/MotorModelDatabase.cpp:44`) — two manufacturers who both make a “D12” cannot coexist — and the import entry points report *net-new* counts, so re-importing a file reports zero (`model/MotorModelDatabase.cpp:61`, pinned as intentional by `tests/MotorDatabasePersistenceTests.cpp:348`).

The query side is a small source-agnostic API: `MotorQuery/MotorSummary` and `listMotors` (`model/MotorModelDatabase.h:33`), filtering in memory on exact manufacturer and impulse class, diameter within ± 0.5 mm (a deliberate tolerance for XML float round-trip fuzz, `model/MotorModelDatabase.cpp:106`), and case-sensitive name substring; results come back name-sorted for free by map iteration (`model/MotorModelDatabase.cpp:98`). `getMotorModel` returns an `std::optional<MotorModel>` by copy (`model/MotorModelDatabase.cpp:81`), via a `find()`-then-`operator[]` double lookup (`model/MotorModelDatabase.cpp:94`).

Persistence is a native XML format (`.qmd` by convention): `saveMotorDatabase` writes a `QtRocketMotorDatabase` root with a `version="0.1"` attribute (`model/MotorModelDatabase.cpp:184`), one motor node per entry carrying every `MetaData` field, delays as a CSV string, and the curve as `thrust time/force` nodes (`model/MotorModelDatabase.cpp:177`); the loader mirrors it with per-field defaults, skips foreign nodes, and recomputes the mass curve on `setMetaData` (`model/MotorModelDatabase.cpp:241`). The round trip over the full bundled import is exact field-by-field (`tests/MotorDatabasePersistenceTests.cpp:297`), resting on `str()`↔`toEnum()` inverses pinned for every enum value (`tests/MotorDatabasePersistenceTests.cpp:245`) — including, notably, misspellings that `CertOrg::str()` emits (“Austrialian...”, “Unkown”, `model/MotorModel.h:128`) and `toEnum` explicitly accepts back (`model/MotorModel.h:143`), so the typos are now a persisted wire format. The `version` attribute is written but never examined on load (`model/MotorModelDatabase.cpp:249`): a future format change has no migration hook.

4.5 Three ingest paths

RockSim .rse — Partial. `RSEDatabaseLoader` reads the XML with `boost::property_tree` and iterates `engine-database.engine-list` (`model/RSEDatabaseLoader.cpp:24`). It honors average/peak thrust, burn time, code (as the common name), the delays CSV, diameter, length, manufacturer, total impulse, and type, converting `initWt/propWt` from grams to kilograms (`model/RSEDatabaseLoader.cpp:81`); impulse class is derived from the code because RSE does not carry it (`model/RSEDatabaseLoader.cpp:73`, handling “1/4A”/“1/2A” prefixes via `model/MotorModel.cpp:97`). What it drops matters more: each `eng-data` sample’s `m` (mass) and `cg` attributes — manufacturer-measured mass and CG histories present in the bundled file (`data/Aerotech.rse:19`) — are ignored in favor of the reconstructed constant- I_{sp} curve of Section 4.2, since the sample loop reads only `t` and `f` (`model/RSEDatabaseLoader.cpp:89`). `Isp`, `massFrac`, throat/exit diameters, and the auto-calc flags are likewise ignored; availability is hardcoded `REGULAR` and the certifying organization `UNK` (`model/RSEDatabaseLoader.cpp:52`). Robustness is uneven: `Type` is the only attribute fetched without a default, so a file missing it throws (`model/RSEDatabaseLoader.cpp:85`), and delay parsing

is unvalidated `atoi` — far laxer than the RASP path below.

RASP .eng — Present. `RASPLoader` is a strict, hand-written validating parser: a seven-field header checked with typed, line-numbered parse errors (`model/RASPLoader.cpp:137`), delays split on `-` with `P/none` mapped to 1000 (the plugged sentinel, `model/RASPLoader.cpp:91`), strictly increasing sample times, non-negative thrust, exactly one mandatory trailing zero-thrust sample, and multi-motor files (`model/RASPLoader.cpp:265`). It inserts an explicit (0, 0) origin when the first sample sits at $t > 0$ (`model/RASPLoader.cpp:207`) and *derives* burn time, total impulse (trapezoidal, `model/RASPLoader.cpp:189`), and average/peak thrust from the samples — which is why RASP motors are immune to the two-clock hazard of [Section 4.3](#). RASP weights are already kilograms, so no conversion applies (`model/RASPLoader.cpp:153`). Rejection behavior is pinned by three negative tests (`model/tests/RASPLoaderTests.cpp:112, 124, 138`).

thrustcurve.org REST — Partial. `ThrustCurveClient` GETs three unauthenticated v1 endpoints — `metadata.json` (`model/ThrustCurveClient.cpp:268`), `search.json` capped at 100 results (`model/ThrustCurveClient.cpp:292`, with truncation detected from the server’s match count, `model/ThrustCurveClient.cpp:312`), and one `download.json` per result (`model/ThrustCurveClient.cpp:248`) — over `utils::CurlConnection`, a single-handle libcurl GET wrapper with 10 s/30 s timeouts and default TLS verification (`utils/CurlConnection.cpp:52`). Download parsing selects exactly one simfile, preferring the first RASP entry with samples and never concatenating alternative series (`model/ThrustCurveClient.cpp:66`); gram-denominated weights are converted to kilograms (`model/ThrustCurveClient.cpp:208`). Rough edges: query values are concatenated into the URL without percent-encoding (`model/ThrustCurveClient.cpp:285`), certifying organization and delays are explicit TODOs (`model/ThrustCurveClient.cpp:197`), and a motor whose download returns no samples is still merged into the database carrying the default zero-thrust curve (`model/ThrustCurveClient.cpp:324`) — it would fly with zero thrust, a case no flight test exercises. The $N+1$ downloads run synchronously on the calling thread, which in the GUI is the event loop; the interface-level consequences are [Section 7](#)’s subject.

The remote source sits behind the `ThrustCurveAPI` interface (`model/ThrustCurveClient.h:68`), and the database exposes a private constructor taking `std::unique_ptr<ThrustCurveAPI>` to a single test-access friend (`model/MotorModelDatabase.h:114`), with the real client lazily constructed on first online use (`model/MotorModelDatabase.cpp:137`). The seam is genuinely consumed: a recording fake drives the online wrappers offline (`tests/MotorDatabasePersistenceTests.cpp:106`), verifying criteria mapping — `nameContains` is deliberately not sent upstream (`model/MotorModelDatabase.h:104`) — and that merged results become visible to local queries (`tests/MotorDatabasePersistenceTests.cpp:539`). The pure response parsers are split from the HTTP layer for the same reason and tested against canned JSON (`model/ThrustCurveClient.h:88`); the HTTP layer itself is not tested ([Section 9](#)).

4.6 The motor in flight

A motor enters the simulation as `model::part::Motor`, a leaf `Part` owning a `MotorModel` by value (`model/parts/Motor.h:28`). Its sole physics contribution is an override: `getMass(t)` forwards to the motor model (`model/parts/Motor.h:40`), so the part tree’s time-aware composite walk ([Section 3](#)) produces $mass(t)$, $CG(t)$, and $I(t)$ with no motor-specific inertia code — the per-unit-mass tensor is a solid cylinder built from the catalog diameter and length, converted from millimeters at the boundary, with a point-mass fallback when geometry is unknown (`model/parts/Motor.cpp:27`). The composite behavior is well pinned: $I(t)$ matches an independent parallel-axis oracle during the burn (`model/tests/MotorTests.cpp:162`), and post-burnout composites freeze bitwise with no recompute (`model/tests/MotorTests.cpp:231`). `setMotorModel` swaps the model in place, re-seeding tensor and

mass and dirty-flagging ancestors so `RocketModel`'s borrowed raw pointer stays valid (`model/parts/Motor.cpp:16`).

Placement is the weak half. `RocketModel::setMotorModel` attaches the single `Motor` child at the airframe root, joining airframe-CM to motor-CM stations with an offset documented as “Zero today” (`model/RocketModel.cpp:124`, `model/RocketModel.h:130`): the motor's mass is coincident with the airframe's center of mass, not seated in an aft motor mount, so the very CG-shift-during-burn machinery the part tree computes so carefully acts at the wrong station. There is no user-facing way to reposition it — a `Motor` cannot be added through the part factory, and a `Motor` node inside a design's part tree is rejected as foreign (`model/DesignSerializer.cpp:157`). The single-motor assumption is explicit: `findMotorInTree` is a DFS for the *first* motor node (`model/RocketModel.cpp:13`), and `Stage.h` is a commented-out “Not yet” include (`model/RocketModel.h:22`) — no clusters, no stages. In flight, thrust participates as `Vector3{0.0, 0.0, motorPart ? motorPart->getMotorModel().getThrust(t) : 0.0}` applied through the CM (`model/RocketModel.cpp:70`) with `getTorques()` returning zero unconditionally (`model/RocketModel.cpp:94`); what that simplification costs is [Section 6](#)'s subject. `launch()` ignites at exactly $t = 0$ (`model/RocketModel.cpp:105`), and `isMotorSet()` is the launch gate both front ends read (`model/RocketModel.h:63`). Designs persist the motor by common name only and re-resolve it against the database on load, warning rather than failing on a miss (`model/DesignSerializer.cpp:271`); the format details belong to [Section 8](#).

Delays and ejection charges deserve their own verdict: **Absent**. Every ingest path parses delay lists (with 1000 as the plugged sentinel, `model/MotorModel.h:303`), the XML round-trips them, and no simulation, GUI, or CLI code ever reads them — there is no ejection event, no recovery deployment, and the only flight event is descending below the launch altitude (`model/RocketModel.cpp:62`). Online-sourced motors do not even populate the field (`model/ThrustCurveClient.cpp:198`). The flight-phase consequences — no deployment drag model, descent simulated as ballistic coast — are taken up in [Section 6](#), and the item is consolidated with the rest of the gap list in [Section 11](#).

4.7 Bundled data

The repository ships exactly one motor data file, `data/Aerotech.rse:1`, whose 252 `engine` entries collapse to 249 stored motors on import: three codes (G77R, G79W, I65W) appear twice and the name-keyed map of [Section 4.4](#) keeps only the last of each, a count the persistence suite's own setup comment acknowledges (`tests/MotorDatabasePersistenceTests.cpp:291`). Nothing is loaded at startup: `QTROCKET_DATA_DIR` is a tests-only compile definition (`tests/CMakeLists.txt:16`), so a GUI or CLI user begins with an empty database and must import a file or search online (`gui/CannonballTab.cpp:125`). A second fixture, `tests/data/motors/estes_small.qmd`, exists purely to exercise the native format. Open-Rocket, by contrast, ships the full `thrustcurve.org` corpus offline; the breadth comparison is drawn in [Section 10](#).

5 The simulation core

The simulation core is where the codebase's central habit — build the seam first, wire in the consumer later — is most visible. What actually flies today is a three-degree-of-freedom point mass: the integrated state is center-of-mass position and velocity, nothing more. Around that modest physics sits a genuinely pluggable machine: two interchangeable ODE solver backends selected by name at runtime, a string-keyed registry of three atmosphere and two gravity models, a hang-proof run loop that maps every possible exit onto a typed verdict, and per-step trajectory statistics. The scaffolding for more — quaternion attitude state, a torque interface, per-step inertia recording, a wind model class — is present, typed, and in several cases tested, but has no consumer. This section reviews what

is integrated, how the loop terminates, how the two solvers differ, what the environment registry offers, and how the Qt-free `QtRocket` controller drives it all.

5.1 The injected ODE and what is actually integrated

The physics enters the solver as a closure built once in the `Propagator` constructor (`sim/Propagator.cpp:36`):

```
std::function<std::pair<Vector3, Vector3>(double, Vector3&, Vector3&)> linearODEs = [this](double t,
↪ Vector3& state, Vector3& rate) -> std::pair<Vector3, Vector3>
{
    Vector3 dPosition;
    Vector3 dVelocity;
    dPosition = rate;
    dVelocity = object->getForces(t, state, rate, *environment) / object->getMass(t);

    return std::make_pair(dPosition, dVelocity);
};
```

This lambda is the entire integrated physics: $\dot{x} = v$, $\dot{v} = F(t, x, v)/m(t)$ over a coupled pair of `Vector3`s. Time is threaded through deliberately, so that the motor thrust curve is sampled at each Runge–Kutta stage’s node time rather than frozen at the step start (`sim/Propagator.cpp:34`, `sim/DESolver.h:41`). The `model::Propagatable` behind `object` is in practice `RocketModel`, the model↔sim bridge described in Section 3.

Environment by injection, not by singleton

The `Propagator` holds the `Environment` and passes `*environment` into every `getForces` call (`sim/Propagator.h:106`, `sim/Propagator.cpp:41`). The force path never reaches back to a global for its physics models, which is what lets tests swap atmospheres and gravity per-run and keeps the `utils`→`controller` back-call cycle out of the dependency graph.

The force model itself (`model/RocketModel.cpp:68`) is three terms: motor thrust applied along world +z regardless of any launch angle (`model/RocketModel.cpp:70`), gravity evaluated at the integrator’s *trial* position so each stage sees a consistent state (`model/RocketModel.cpp:77`), and quadratic drag with a single user-set scalar C_d and reference area (defaults $C_d = 1.0$, $A = 1.134 \times 10^{-3} \text{ m}^2$; `model/RocketModel.h:138`):

```
// Clamp altitude to >= 0: trial states crossing z=0 fall outside the atmosphere models' domain.
const double altitude = position[2] > 0.0 ? position[2] : 0.0;
const double rho = atmosphere->getDensity(altitude);
const double speed = velocity.norm();
const Vector3 drag = -0.5 * rho * speed * dragCoefficient * referenceArea * velocity;
```

The $|v| \cdot v$ formulation gives zero drag at $v = 0$ with no division, and the altitude clamp exists because the table-backed atmosphere throws `std::out_of_range` below its lowest bin (`model/RocketModel.cpp:84`, `utils/Bin.cpp:51`). What this force model does *not* contain — angle of attack, normal force, a stability moment, the Barrowman machinery that the model layer already composes but the force path never reads (`model/Propagatable.h:61`) — is the subject of Section 6.

The six-degree-of-freedom scaffolding deserves explicit accounting because it is easy to mistake for capability. `StateData` carries orientation and orientation-rate quaternions (initialized to the degenerate all-zero quaternion, not identity), a DCM, and 3-2-1 Euler angles (`sim/StateData.h:46`); a repository-wide search finds no code that writes or reads any of those fields. The orientation

integrator is a commented-out member with a design note about the future `DESolver<Quaternion>` ODE (`sim/Propagator.h:103`), and `RocketModel::getTorques` returns `Vector3{0.0, 0.0, 0.0}` (`model/RocketModel.cpp:94`). One piece of the seam is already live: every recorded step snapshots composite mass, CG, and inertia via `writeMassProperties` (`model/RocketModel.cpp:53`), with the header naming these fields “the seam the 6-DOF rotational ODE will read” (`sim/StateData.h:59`). Attitude propagation is therefore **Absent** at commit 254cd41, but the interfaces it will need are in place and typed.

5.2 The run loop and typed termination

`Propagator::runUntilTerminate` (`sim/Propagator.cpp:58`) begins every run clean — a fresh `Nominal` verdict and zeroed statistics (`sim/Propagator.cpp:63`) — then loops: read the current state, take one integrator step, and run a guard cascade (Fig. 2). The first guard rejects a non-finite state *before* it is recorded or read by the termination predicate, because a NaN coordinate fails every comparison and the $z < 0$ test would never trip (`sim/Propagator.cpp:94`). The second is the launch-pad hold: thrust curves begin with an implicit (0, 0) sample, so over the first step a from-rest rocket sees essentially zero thrust, accelerates downward, and would cross $z < 0$ and terminate after one step. Until the rocket first achieves $z > 0$ under power it is therefore clamped to $z = 0$ at rest — physically, the pad’s normal force cancels the downward force (`sim/Propagator.cpp:105`). A rocket that can never lift stays put with `maxAltitude = 0`, which is what arms the `NoLiftoff` guard instead of letting the run end silently.

The five `TerminationReason` values (`sim/Propagator.h:35`) partition every exit:

Nominal the model’s own predicate fired. `RocketModel` defines end of flight as *descending below the launch site*: `position[2] < 0.0 && velocity[2] < 0.0` (`model/RocketModel.cpp:62`). There are no recovery or staging events; every flight ends ballistic at that crossing.

NoLiftoff more than 3 s elapsed without the peak altitude exceeding 1 m (thresholds at `sim/Propagator.h:26`; guard at `sim/Propagator.cpp:146`) — an underpowered motor or a hover-balance flight.

NonFiniteState NaN/Inf state caught before recording, so the poisoned sample never enters the history (`sim/Propagator.cpp:97`).

MaxSimTimeExceeded the hard backstops: simulated time past `maxSimTime` (default 7200 s, settable; `sim/Propagator.h:115`) or 10^8 iterations (`sim/Propagator.h:28`).

IntegratorError the whole loop is wrapped in try/catch, so an RKF45 step-size underflow throw becomes a verdict instead of escaping (`sim/Propagator.cpp:81`, `sim/Propagator.cpp:169`).

Iteration-cap abort is misreported

The sim-time cap and the 10^8 -iteration cap share one branch and one verdict: `if(currentTime > maxSimTime || ++iterations >= maxIterations)` sets `MaxSimTimeExceeded` (`sim/Propagator.cpp:157`), and the enum has no distinct iteration-cap value (`sim/Propagator.h:40`). A run killed by adaptive step collapse — millions of vanishingly small steps at low simulated time — reports a time-cap exceedance even when simulated time is nowhere near `maxSimTime`. The abort is correct; the diagnosis is wrong. **FAIL**

Recording is gated by a `saveStates` flag: when saving, each accepted step snapshots mass properties and appends (`time`, `StateData`) to the history stored on the `Propagatable` (`sim/Propagator.cpp:128`, `model/Propagatable.h:47`). Two details matter downstream. First, the timestamp convention lags one step: `appendState(currentTime, nextState)` records the state reached at $t + h$ against timestamp t , because `currentTime` advances by the step actually taken only *after* recording (`sim/Propagator.cpp:133`,

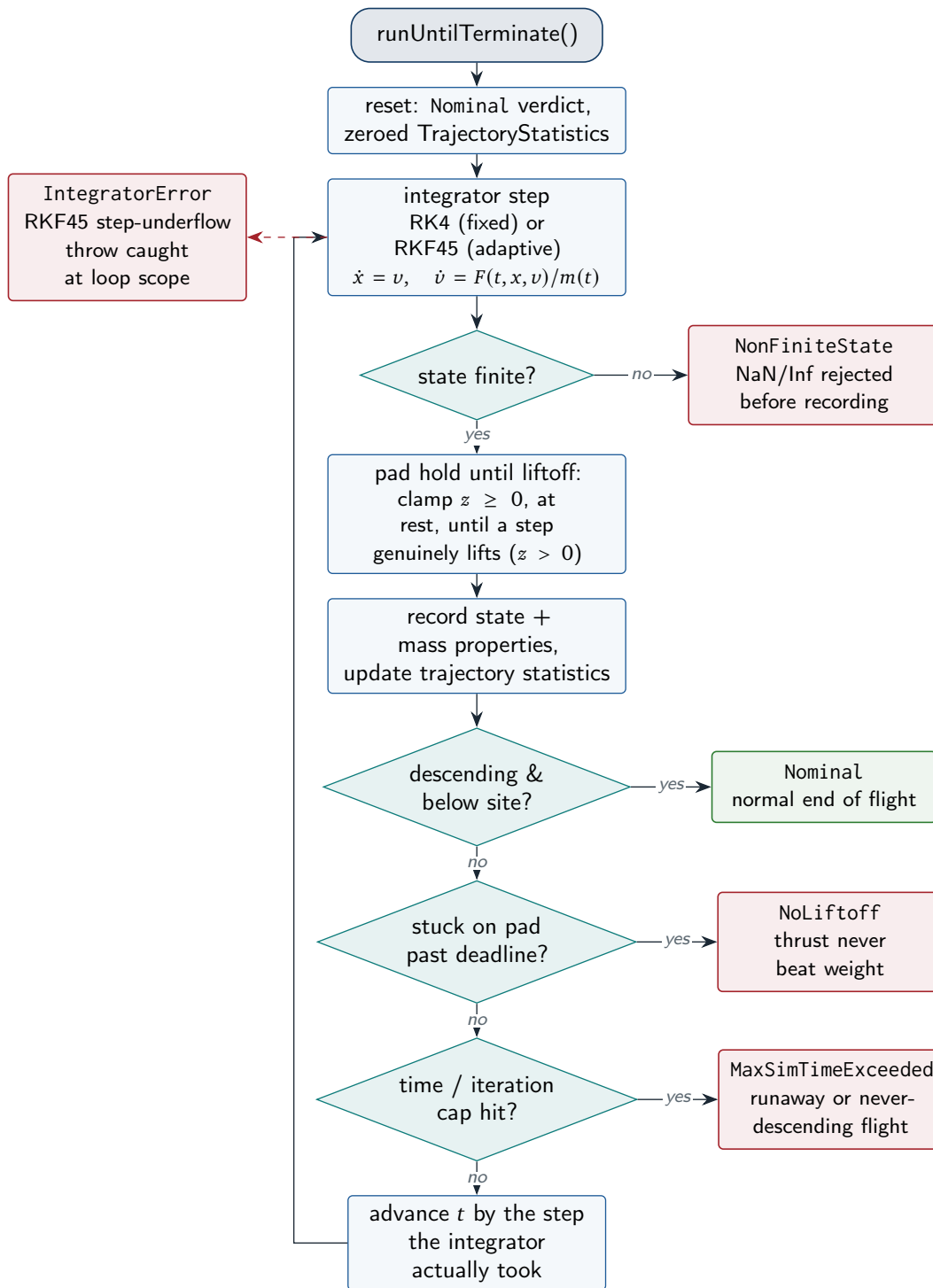


Figure 2. The `runUntilTerminate` flight loop (`sim/Propagator.cpp:58`): one integrator step, a finiteness check, the launch-pad hold, state recording, and the guard cascade that maps every exit onto one of the five typed termination reasons. The `IntegratorError` path is the loop-scope catch that converts an RKF45 step-size underflow throw into a verdict.

`sim/Propagator.cpp:166`). Every consumer of the history — plots, CSV export, apogee times — inherits the offset; a propagator test widens its apogee-time tolerance to $3dt$ and names “the recorder’s one-step time-stamp offset” rather than pinning the convention as intended (`tests/PropagatorTests.cpp:181`). Second, the `saveStates=false` path the loop carefully supports is dead: `retainStates` has no callers anywhere (`sim/Propagator.h:62`).

`TrajectoryStatistics` is updated on every step regardless of `saveStates`, so the flight summary and the no-liftoff guard work even without a retained history (`sim/Propagator.cpp:127`). The fold uses strict $>$ comparisons — first attainment wins, and NaN samples are ignored by construction — tracking peak altitude and its time, peak speed ($|\nu|$ norm) and its time, and total flight time as the last sample time (`sim/TrajectoryStatistics.h:26`).

5.3 The Integrator facade and the two solver backends

Both solvers derive from `DESolver<T>` (with `T` restricted to `Vector3` or `Quaternion` by `static_assert`; `sim/RK4Solver.h:33`, `sim/RK45Solver.h:51`) and advance the coupled (state, rate) pair, returning a `StepResult{state, rate, stepSize}` so the caller advances its clock by the step the solver *actually* took (`sim/DESolver.h:21`). The `Quaternion` instantiation is never used — another finished seam with zero consumers.

`RK4Solver` is textbook fixed-step RK4 with stages at $t, t + dt/2, t + dt/2, t + dt$ (`sim/RK4Solver.h:45`); its timestep starts as NaN, and calling `step()` before `setTimeStep()` is a logged error returning a zero result rather than undefined behavior (`sim/RK4Solver.h:49`). `RK45Solver` implements the classical Fehlberg tableau — six stages with the Butcher coefficients as `constexpr` arrays (`sim/RK45Solver.h:169`), each stage evaluated at its node time $t + C_n h$ (`sim/RK45Solver.h:92`) so a thrust transient is visible to the error estimate. The controller is a single absolute norm over state and rate, $\text{err} = \sqrt{\|y_5 - y_4\|_{\text{state}}^2 + \|y_5 - y_4\|_{\text{rate}}^2}$ (`sim/RK45Solver.h:120`): a rejected step shrinks h by $(\text{tol}/\text{err})^{1/4}$ floored at $0.1\times$ (`sim/RK45Solver.h:124`); an accepted step grows the next guess by $(\text{tol}/\text{err})^{1/5}$ capped at $5\times$ (`sim/RK45Solver.h:132`); below $h_{\min} = 10^{-15}$ it throws (`sim/RK45Solver.h:80`); and the fifth-order estimate is returned as the result (`sim/RK45Solver.h:150`).

The vacuum step-runaway clamp

The next-step guess is clamped to $h_{\max}$, by default $10\times$ the seeded timestep (`sim/RK45Solver.h:144`, `sim/RK45Solver.h:165`). This is not a tuning nicety but a correctness requirement peculiar to embedded-pair steppers: constant-acceleration vacuum coasting is integrated *exactly* by both embedded orders, so $\text{err} \approx 0$ every step, the $5\times$ growth applies indefinitely, and the integrator leaps past apogee and the ground in a few giant steps. The failure was real and is now regression-guarded (`sim/tests/RK45SolverTests.cpp:75`).

The adaptive solver’s tuning surface stops at its own header. `setErrorTolerance` and `setMaxStepSize` exist on the concrete class (`sim/RK45Solver.h:70`), but `Integrator` stores its solvers as `unique_ptr<DESolver<Vector3>>` (`sim/Integrator.h:86`) and exposes neither, and neither do `Propagator`, `QtRocket`, the GUI, or the CLI. Users get the hardcoded 10^{-6} absolute tolerance (`sim/RK45Solver.h:46`) and the $10\times$ -seed $h_{\max}$, full stop; the header itself flags the unscaled absolute norm as “a possible refinement” (`sim/RK45Solver.h:36`).

Selection is a string-keyed facade: `Integrator` maps Runge-Kutta 4th Order (the default) and Runge-Kutta-Fehlberg to freshly constructed solvers on each `setIntegratorModel` call, and an unknown name is a *logged* no-op that keeps the current model usable (`sim/Integrator.h:52`). The name travels from the GUI’s integrator combo (populated from `getAvailableIntegratorModels()`;

gui/SimOptionsTab.cpp:74) or the CLI `setintegrator` command (cli/Repl.cpp:647) through `QtRocket::setIntegratorModel` (QtRocket.h:36) to `Propagator::setIntegratorModel` (sim/Propagator.h:90). The facade has one sharp edge: `setIntegratorFunction` only stores the `lambda` (sim/Integrator.h:72), so the active solver keeps whatever function it captured at construction — the constructor even builds its default RK4 with a null ODE function (sim/Integrator.h:28) — and a freshly selected solver needs its timestep re-seeded. The required call order is documented in a test rather than enforced by the API (sim/tests/IntegratorTests.cpp:36); `Propagator` gets it right and re-asserts the configured step at the top of every run (sim/Propagator.cpp:60).

5.4 The Environment registry

`Environment` is a string-keyed registry constructed with `Constant Gravity` and `Constant Atmosphere` defaults (sim/Environment.h:31). All of its models are genuinely runtime-selectable from both front ends — GUI combos populated from the registries (gui/SimOptionsTab.cpp:72) and the CLI `setatmosphere/setgravity` commands (cli/Repl.cpp:591, cli/Repl.cpp:618). Table 3 summarizes the inventory.

Family	Registry key	Behavior	Status
Atmosphere	<code>Constant Atmosphere</code>	sea-level ISA constants at every altitude ($\rho = 1.225$, $p = 101325$, $T = 288.15$; sim/ConstantAtmosphere.h:15)	Present
Atmosphere	<code>US Standard 1976</code>	seven-layer USSA76 closed forms, 0–80 km validated (sim/USStandardAtmosphere.cpp:93)	Present
Atmosphere	<code>Vacuum</code>	all properties zero; the drag-free baseline (sim/VacuumAtmosphere.h:17)	Present
Gravity	<code>Constant Gravity</code>	(0, 0, −9.80665) (sim/ConstantGravityModel.h:18)	Present
Gravity	<code>Spherical Gravity</code>	Newtonian inverse-square over the geoid ground radius (sim/SphericalGravityModel.cpp:28)	Present
Geoid	(fixed member)	WGS84 mean radius for any lat/lon (sim/SphericalGeoidModel.cpp:18)	Partial
Wind	(not registered)	zero-vector stub, zero call sites (sim/WindModel.cpp:16)	Absent

Table 3. The `Environment` inventory at commit 254cd41. The three atmospheres and two gravity models are live and selectable from both front ends; the geoid is a single hardwired member; the wind class is compiled but unreachable.

The US Standard Atmosphere is the core’s showpiece of physical fidelity: seven layers with bases at 0/11/20/32/47/51/71 km held in static `utils::Bin` tables of base temperature, density, pressure, and lapse rate (sim/USStandardAtmosphere.cpp:21), evaluated in closed form within each layer — the exponential barometric formula in isothermal layers and the power-law form with exponent $gM/(R*L)$ in gradient layers (sim/USStandardAtmosphere.cpp:93) — rather than by table interpolation. Density, pressure, and temperature are validated against published NOAA report values to 0.1% (most rows) and 0.5% (a few density rows) up to 80 km (sim/tests/USStandardAtmosphereTests.cpp). Speed of sound ($\sqrt{\gamma R*T/M}$) and Sutherland viscosity are implemented (sim/USStandardAtmosphere.cpp:143) but sit unconsumed: the force path samples only density, and no test asserts either property for this model — the only speed-of-sound assertions anywhere target the `Constant` and `Vacuum` models. The edges are soft in both directions: below 0 m the `Bin` lookup throws (guarded by the force-path altitude clamp; utils/Bin.cpp:51), while above 71 km the last layer’s formula extrapolates indefinitely because falling off the table’s end returns the last bin (utils/Bin.cpp:57) — the real USSA76 regime change above 80 km is not modeled, and the header still carries an unresolved verification TODO (sim/USStandardAtmosphere.h:18).

Spherical gravity is a true Newtonian inverse square: the launch frame maps to geocentric coordinates as $(x, y, z + \text{groundLevel})$ and $a = -GM\vec{r}/|\vec{r}|^3$ with $GM = 3.986004418 \times 10^{14}$ (`sim/SphericalGravityModel.cpp:28`, `utils/math/Constants.h:26`). Its geoid, however, is a placeholder twice over: the single `SphericalGeoidModel` accepts latitude and longitude and ignores both, returning the WGS84 mean radius 6371008.8 m unconditionally (`sim/SphericalGeoidModel.cpp:18`, `utils/math/Constants.h:24`), and the gravity model queries it once at construction with literal $(0, 0)$ — the comment reads “ $(0, 0)$ is a placeholder until a real launch site is plumbed in” (`sim/SphericalGravityModel.cpp:19`). There is no launch-site concept anywhere in `Environment` or the controller, so altitude above the pad is implicitly altitude above sea level in every atmosphere lookup.

One registry behavior is inconsistent with the integrator facade and worth stating plainly: `setGravityModel` and `setAtmosphereModel` have no else branch, so an unrecognized name is a *silent* no-op (`sim/Environment.h:60`) — tests pin the no-op (`sim/tests/EnvironmentTests.cpp:94`) — whereas the same mistake on `setIntegratorModel` is logged as an error (`sim/Integrator.h:64`). Today both front ends populate their choices from the registries themselves, so a typo cannot arise; a future config file or save format would make it invisible.

Wind is the purest specimen of the seam-without-consumer motif. The class exists and compiles into the sim library (`sim/CMakeLists.txt:24`); its entire behavior is

```
Vector3 WindModel::getWindSpeed(double /* x */, double /* y */, double /* z */)
{
    return Vector3(0.0, 0.0, 0.0);
}
```

(`sim/WindModel.cpp:16`), and a repository-wide search finds zero call sites: it is never instantiated, has no slot in the `Environment` registry (`sim/Environment.h:103`), and never enters `getForces` — drag uses ground-relative velocity only (`model/RocketModel.cpp:88`). Wind cannot affect any simulation at this commit; what that costs in flight fidelity is discussed in [Section 6](#).

5.5 The QtRocket controller

`QtRocket` is the Qt-free controller singleton whose ownership, API surface, and racy double-checked initialization are examined in [Section 2](#); what matters here is how it runs a flight. `launchRocket()` (`QtRocket.cpp:54`) runs the four launch steps of [Section 2](#) synchronously, end to end, on the calling thread (`RocketModel::launch` copies the initial state to current and ignites at $t = 0$; `model/RocketModel.cpp:105`). There is no worker thread, no progress reporting, and no cancellation: the simulation runs to completion on whichever thread calls it — the Qt event thread when the GUI’s launch button fires (`gui/CannonballTab.cpp:116`), so the UI freezes for the run’s duration, and the REPL loop for the CLI (`cli/Repl.cpp:688`). Nothing reads the state history concurrently today and no lock protects it. The GUI’s own threading arrangement — the entire `QApplication` runs on a spawned `std::thread` while `main()` blocks on `join` (`gui/GuiRunner.cpp:62`) — is examined in [Section 7](#).

Results flow back by const reference: `getStates()` exposes the `(time, StateData)` history stored on the `Propagatable` (`model/Propagatable.h:47`), alongside `getTrajectoryStatistics()` and `getTerminationReason()` (`QtRocket.h:48`). Two loose ends round out the picture. `addRocket` — the API that would swap in a fresh model/propagator pair — has no call sites (`QtRocket.h:41`), joining `retainStates` in the dead-API column, and `setInitialState`’s doc comment still describes a stale twelve-element pitch/yaw/roll state vector that the 3-DOF core never consumes (`QtRocket.h:66`). Both are consolidated with the other incompleteness findings in [Section 11](#).

6 Aerodynamics and flight-physics fidelity

This is the keystone section of the review, because it is where the document’s recurring motif is sharpest. QtRocket’s flight physics splits cleanly into two halves that share a codebase but not a call graph: a small, honest, well-guarded three-degree-of-freedom point-mass model that every flight actually runs, and a substantially more ambitious aerodynamic layer — per-part Barrowman normal-force slopes, a composite center of pressure on the CG’s own datum, per-part drag fields, wind, torques — that is built, unit-tested, and consumed by nothing. Figure 3 shows both halves at once; this section describes each precisely, then the latent defects that must be repaired before the dormant half can be wired in, and finally the flight-envelope features for which no machinery exists at all.

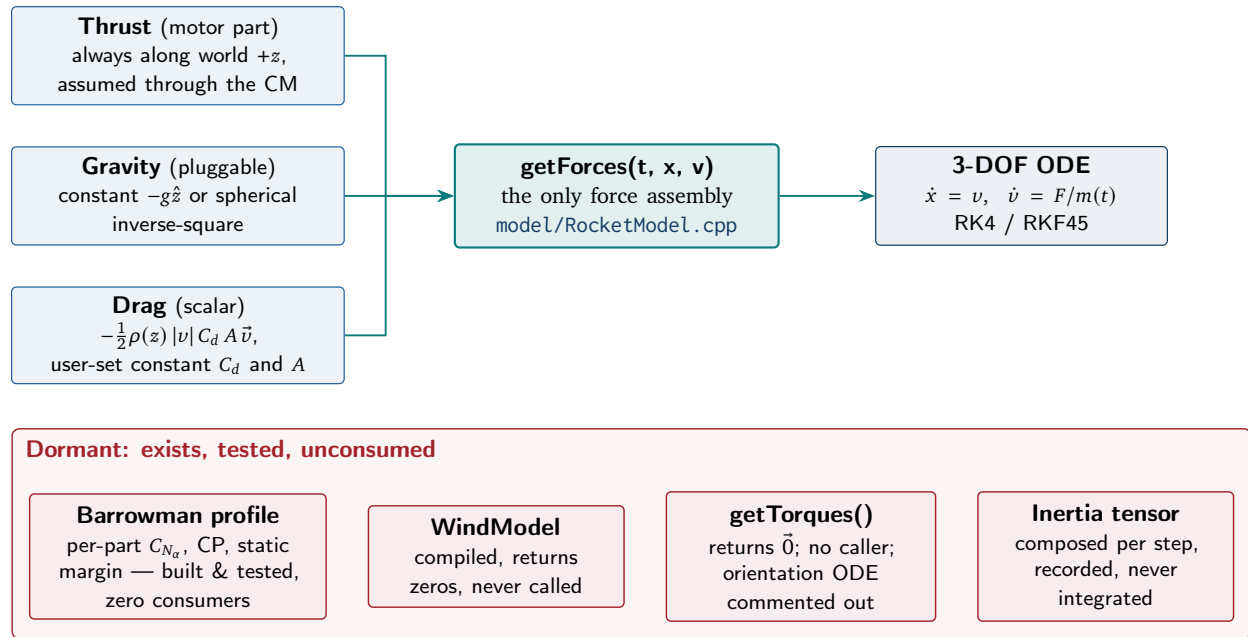


Figure 3. The flight-physics force path at commit 254cd41. Three terms — world-vertical thrust, pluggable gravity, and a single quadratic drag law — are assembled by `getForces()` and integrated as three linear degrees of freedom. The red-edged layer below is built and tested but has no production caller.

6.1 The consumed force path

The single production force model is `RocketModel::getForces()` (`model/RocketModel.cpp:68`), taking the trial position and velocity plus the environment, and invoked from the ODE lambda the propagator builds at construction: $\dot{v} = F(t, x, v)/m(t)$ with the time-varying mass supplied by the burning motor (`sim/Propagator.cpp:36`). Exactly three terms are summed.

Thrust. Applied along world +z for the entire burn, “assumed through the CM” (`model/RocketModel.cpp:70`). The comment says “along the rocket’s z-axis,” but since no attitude state exists anywhere in the dynamics, the rocket’s z-axis is the world z-axis, permanently.

Gravity. `gravityModel->getAccel(position) * getMass(t)`, evaluated at the integrator’s trial position so every Runge–Kutta stage sees a consistent state (`model/RocketModel.cpp:77`); the model is pluggable between constant and spherical inverse-square via `sim::Environment` (`sim/Environment.h:60`).

Drag. A single quadratic point-mass term opposing the full velocity vector, with a user-set constant drag coefficient and a user-set constant reference area (`model/RocketModel.cpp:88`):

```
// Clamp altitude to >= 0: trial states crossing z=0 fall outside the atmosphere models' domain.
const double altitude = position[2] > 0.0 ? position[2] : 0.0;
const double rho = atmosphere->getDensity(altitude);
const double speed = velocity.norm();
const Vector3 drag = -0.5 * rho * speed * dragCoefficient * referenceArea * velocity;
```

Listing 5. The entire production drag model (`model/RocketModel.cpp:84`). One constant C_d , one constant reference area; the $|v|\vec{v}$ form gives zero drag at rest with no division, and the altitude clamp keeps trial states below ground inside the atmosphere models' domain.

`dragCoefficient` defaults to 1.0 (`model/RocketModel.h:138`) and `referenceArea` to $1.134 \times 10^{-3} \text{ m}^2$ — a 38 mm disc (`model/RocketModel.h:141`). Density comes from the pluggable `AtmosphericModel`: constant 1.225 kg/m^3 (`sim/ConstantAtmosphere.h:15`), a genuinely layered US Standard 1976 implementation with lapse-rate and isothermal branches (`sim/USStandardAtmosphere.cpp:93`), or vacuum (`sim/VacuumAtmosphere.h:17`); the surrounding integration loop, its guards, and the environment plumbing are [Section 5](#)'s subject. This consumed path is the best-tested physics in the tree: an analytic terminal-velocity force balance is derived from the live models and checked for sign and magnitude (`tests/PhysicsIntegrationTests.cpp:246`), and drag-versus-vacuum sweeps run across every reference airframe and the full 1/4A–M motor ladder under both integrators (`tests/DesignMatrixTests.cpp:524`).

Two structural consequences of this force path deserve plain statement. First, *launch angle is a velocity, not an attitude*. Both front ends implement an angled launch by splitting the user-supplied initial speed into v_x/v_z components — the GUI at `gui/CannonballTab.cpp:105`, the CLI's launch handler consuming the `setangle`-stored value at `cli/Repl.cpp:680` — and nothing else in the dynamics knows the angle exists. With zero initial speed, setting a launch angle changes nothing at all. With nonzero initial speed, thrust is permanently misaligned with the velocity vector for the whole burn — there is no gravity turn and no thrust-along-body-axis, so the model is physically consistent only for vertical flights. The one angled-launch test asserts only the kinematic downrange displacement of the tilted initial velocity (`tests/PhysicsIntegrationTests.cpp:182`).

Second, the reference area that the drag law multiplies is never connected to the rocket's geometry, despite the geometry-derived value existing and being correct.

Geometry-derived reference area never reaches the drag law

`RocketModel::deriveReferenceAreaFromGeometry()` (`model/RocketModel.cpp:48`) implements the max-frontal-disc convention over the part tree via `Part::maxFrontalReferenceArea()` (`model/parts/Part.cpp:251`), with fin sets deliberately reporting the body disc rather than tip extent (`model/parts/FinSet.h:71`) — the same convention OpenRocket uses ([Section 10](#)). It has no production caller. The drag law reads the stored `referenceArea` scalar; the header's promise that “the geometry default applies only when it [the override] hasn't” (`model/RocketModel.h:76`) is implemented by nothing; and design-file load applies the saved area only when `referenceAreaOverridden="true"` (`model/DesignSerializer.cpp:265`). All 25 bundled reference designs say “false”, so every one of them — including the 75 mm x175 airframe — flies with whatever area was last set, by default the 38 mm disc. Drag therefore does not scale with airframe class, and the sweep-test comment claiming the “design's own geometry-derived reference area ... is used” (`tests/DesignMatrixTests.cpp:518`) does not match the code. **FAIL**

6.2 The dormant Barrowman layer

Alongside the scalar drag law sits a real stability-analysis seam. Its value types live in `sim/Aero.h`: `AeroComponent` carries `cnAlpha` (the normal-force-coefficient slope per radian, normalized to a shared reference area), `cnAlphaXcp` (the slope-weighted CP moment, stored as a product so zero-lift parts drop out of the weighted average), and `cd` (`sim/Aero.h:32`); `AeroProfile` folds components additively and exposes $x_{cp} = \sum C_{N_\alpha} x_{cp} / \sum C_{N_\alpha}$ behind a `cpValid` guard (`sim/Aero.h:51`).

The per-part mathematics is genuine Barrowman, not placeholder. The conical nose cone reports $C_{N_\alpha} = 2$ referenced to its base area and rescaled by $\pi R^2 / A_{\text{ref}}$, with the CP at the textbook two-thirds of the length aft of the tip, converted to the part’s own CM datum (x_{cp} from $CM = L/3 - \bar{h}$: $+L/12$ for a solid cone, 0 for a shell) (`model/parts/ConicalNoseCone.cpp:64`) — self-consistent with the documented $+z = \text{forward}$ part frame in which each part spans $z \in [-L, 0]$ (`model/parts/Part.h:130`). The trapezoidal fin set implements the standard N -fin term with the mid-chord line l_m and the fin-body interference factor $K_{fb} = 1 + r_b / (s + r_b)$, referenced to the body disc and rescaled (`model/parts/FinSet.cpp:77`), with the degenerate on-axis case $r_b = 0$ guarded rather than allowed to go singular (`model/parts/FinSet.cpp:63`) and the fin CP taken from the root leading edge by the usual trapezoid formula (`model/parts/FinSet.cpp:84`). A constant-diameter body tube correctly contributes zero normal force (`model/parts/BodyTube.cpp:38`), and the base-class default is aerodynamically inert (`model/parts/Part.h:117`), so motors and placeholder parts drop out.

Composition is a pure-geometry pass, separate from the time-aware mass walk: `Part::getCompositeAero(refArea)` walks the resolved placement cache and re-expresses every part’s CM-datum x_{cp} onto the tree-root tip datum (`model/parts/Part.cpp:231`) — the same datum as the composite CM.

The static margin is a subtraction by construction

Because `getCompositeAero` and `getCompositeCm` share one datum, the composite `cp()` and `cg()` are directly comparable and the static margin is literally `cp() - cg()` — the design intent is stated in the seam’s own documentation (`sim/Aero.h:25`). The moment-weighted storage (`cnAlphaXcp` as a product) makes composition pure addition, and unit tests pin the datum correctness across nested trees (`model/tests/NoseConeTests.cpp:175`, `tests/PlacementInvarianceTests.cpp:106`). The machinery for a live CP/CG/stability readout exists end to end — except for a caller.

And that is the finding: none of it is consumed. `getCompositeAero` has zero production call sites — its callers are three test files, and the aero suite’s own header says so in as many words: “NOTE: nothing in production consumes this in P2 – these tests pin the seam for P5” (`model/tests/AeroTests.cpp:18`). The `sim::Aero` `aeroData` member on the model-sim bridge is a default-constructed, admittedly unread back-compatibility shim (`model/Propagatable.h:61`, `sim/Aero.h:63`). No angle of attack is computed anywhere, no normal force or restoring moment ever acts on the trajectory, and no stability margin — in meters or in calibers — is surfaced by the GUI or the CLI; the word “caliber” appears nowhere in the tree, and the placement-invariance suite deliberately excludes static margin from its legacy-baseline comparison (`tests/PlacementInvarianceTests.cpp:318`). Status: **Partial** — the mathematics is present and tested; the flight it should inform is untouched by it.

6.3 Latent defects that must precede consumption

Wiring the Barrowman layer into flight is not purely additive work; one verified frame defect sits inside the fin set, and it is the kind that a future consumer would inherit silently because the test suite currently enforces it.

The fin set's chordwise frame is mirrored

The documented part frame is $+z$ =forward (`model/parts/Part.h:130`); the cone converts its base-relative centroid into that frame correctly (`model/parts/ConicalNoseCone.cpp:51`), and the visualizer draws the fin root leading edge at the fore plane, sweeping the tip aft (`visualizer/RocketMesh.cpp:350`). The fin set's mass and aero math, however, uses aft-positive from-root-LE distances directly as $+z$ offsets: `finSetCmOffset` returns $x_c - c_r/2$ where the LE-forward frame implies $c_r/2 - x_c$ (`model/parts/FinSet.cpp:57`), and the CP conversion in [Listing 6](#) has the same mirrored sign. The two errors do not cancel when `getCompositeAero` threads the CM offset (`model/parts/Part.cpp:244`): the composite fin CP lands $c_r - 2x_{cp,LE}$ aft of truth (25.6 mm for the reference test fin with $c_r = 100$ mm, $c_t = 50$ mm, sweep 40 mm; half a chord for an unswept rectangular fin), and the fin-set mass sits $2x_c - c_r$ forward of truth (13.3 mm for that fin) in the composite CM and inertia. The unit tests pin the mirrored values as literal regression anchors (`model/tests/FinSetTests.cpp:207`), so the suite enforces the defect rather than detecting it. **FAIL**

```
// Barrowman fin CP, from the root LE, converted to the set's own CM (the composite datum):
// x_cp(from CM) = x_cp(from root LE) - (axial mass centroid from root LE).
const double xcpFromRootLE = (sweep / 3.0) * (rootChord + 2.0 * tipChord) / sumc
    + (1.0 / 6.0) * (sumc - rootChord * tipChord / sumc);
const double xcMass = (rootChord * rootChord + rootChord * tipChord + tipChord * tipChord
    + sweep * (rootChord + 2.0 * tipChord)) / (3.0 * sumc);
const double xcpFromCm = xcpFromRootLE - xcMass;
return {cnAlpha, cnAlpha * xcpFromCm, 0.0};
```

Listing 6. The fin CP conversion (`model/parts/FinSet.cpp:84`). Both `xcpFromRootLE` and `xcMass` are aft-positive distances from the leading edge, used directly as offsets in the $+z$ =forward frame — internally consistent only if the fin flew leading-edge-aft.

The consequences split by time horizon. The mass half is live *today*: every swept-fin design carries its fin mass slightly forward of truth in the composite CM and inertia tensor — small in absolute terms because fins are light, but a real error in the mass properties the integrator consumes. The CP half is latent only because nothing consumes the aero profile; once something does, the aft-displaced fin CP would *inflate* the apparent static margin — fins dominate the aft CP, so the error direction is non-conservative for exactly the stability readout the seam was built to provide. A closely related dormant-frame assumption — the composite-inertia walk combining child tensors without the RIR^T rotation, identity-only today and flagged in the code (`model/parts/Part.cpp:220`) — belongs to the part-tree discussion in [Section 3](#).

6.4 Degrees of freedom: the rotational half does not exist

The 3-DOF/6-DOF question has a crisp answer at this commit: the simulator is three-linear-DOF in fact, not merely in configuration. `getTorques()` exists on the `Propagatable` bridge interface (`model/Propagatable.h:30`), the rocket's override returns the zero vector (`model/RocketModel.cpp:96`), and — decisively — it has no caller anywhere in the simulation loop; the only other implementation is a test mock, also zero (`tests/PropagatorTests.cpp:43`). The orientation integrator is a commented-out member sitting beside the live linear one ([Listing 7](#)). `StateData` carries the full 6-DOF scaffolding — orientation and orientation-rate quaternions, a direction-cosine matrix, and Euler angles (`sim/StateData.h:45`) — none of which is ever written by any code, and the quaternions are zero-initialized to (0, 0, 0, 0), which is not the identity rotation or any rotation at all (`sim/StateData.h:46`); the first real

consumer inherits an invalid orientation plus a documented convention hazard, since the member’s “(vector, scalar)” annotation contradicts Eigen’s (w, x, y, z) constructor order. The scaffolding is nonetheless deliberate: the composite mass, CG, and inertia tensor are recorded into every state sample expressly as “the seam the 6-DOF rotational ODE will read” (`sim/StateData.h:59`), while the integrator advances only position and velocity (`sim/Propagator.cpp:36`).

```
std::unique_ptr<sim::Integrator> linearIntegrator;
// The 6-DOF orientation integrator will use the same DESolver<Quaternion> interface, with a
// time-keyed ODE callback so getTorques() can be sampled at each stage's node time like getForces().
// std::unique_ptr<sim::RK4Solver<Quaternion>> orientationIntegrator;
```

Listing 7. The clearest single statement of the 3-DOF reality: the orientation integrator is a pre-designed, commented-out member beside the live linear one (`sim/Propagator.h:100`).

6.5 The rest of the flight envelope

Beyond attitude, several envelope features have either dead code or no code. Each is presented here as a finding; severity ranking and consolidation belong to [Section 11](#).

Wind: dead code. Absent `sim::WindModel` is compiled into the `sim` library (`sim/CMakeLists.txt:24`) and shaped as a virtual extension point (`sim/WindModel.h:16`), but its one implementation returns $(0, 0, 0)$ with all three position arguments ignored (`sim/WindModel.cpp:16`), and no other file in the repository references it: `Environment` has no wind slot (`sim/Environment.h:101`) and `getForces` takes `velocity.norm()` directly (`model/RocketModel.cpp:87`). Relative airspeed therefore equals ground velocity by construction — and even with wind plumbed in, weathercocking would remain impossible without the attitude and normal-force machinery of [Sections 6.2](#) and [6.4](#).

Flight phases and recovery: nothing changes at apogee. Absent The only flight-phase logic in the simulator is the terminate predicate — descending below the launch site, `position[2] < 0.0` `&& velocity[2] < 0.0` (`model/RocketModel.cpp:65`). `getForces` has no apogee or phase branch (`model/RocketModel.cpp:68`); there is no deployment event, no parachute or streamer, no $C_d A$ switch, and descent is ballistic under the same constant drag law that governed ascent. Apogee exists only as a running maximum in the trajectory statistics (`sim/TrajectoryStatistics.h:29`). Motor ejection delays are parsed from RSE files (`model/RSEDatabaseLoader.cpp:59`) and stored on the motor model (`model/MotorModel.h:303`) but consumed by no flight logic — and the `thrustcurve.org` import does not populate them at all (`model/ThrustCurveClient.cpp:198`); the motor-side detail is [Section 4](#)’s.

Per-part drag: the field exists, every value is zero. Partial `AeroComponent.cd` was designed for an additive per-part drag build-up normalized to the shared reference area (`sim/Aero.h:36`), and the composite sums it — but every concrete part returns $c_d = 0$: the body tube’s comment says “`cd` stays 0 – skin friction over `getWettedArea()` is its eventual contribution” (`model/parts/BodyTube.cpp:41`), and the cone and fin set return literal zeros (`model/parts/ConicalNoseCone.cpp:67`, `model/parts/FinSet.cpp:89`). Wetted-area getters exist for the purpose (`model/parts/BodyTube.h:51`, `model/parts/FinSet.h:70`) with no consumer. There is no skin-friction, base-drag, or pressure-drag decomposition of any kind.

Launch rod or rail: only a pad clamp. Partial The propagator holds the rocket at $z = 0$ with zero velocity until a step genuinely lifts it (`sim/Propagator.cpp:111`) and aborts a flight that never does (`sim/Propagator.cpp:146`) — useful guards, described with the rest of the loop in [Section 5](#). But there is no guided phase: no rod length, no rod-departure velocity, no direction constraint

— and since attitude does not exist, a rod could not constrain the thrust direction anyway.

Mach and Reynolds dependence: computed inputs, no consumer. Absent The atmosphere interface exposes `getSpeedOfSound` and `getDynamicViscosity` (`sim/AtmosphericModel.h:17`), and the US Standard 1976 model implements both correctly ($\sqrt{\gamma R^* T/M}$ at `sim/USStandardAtmosphere.cpp:143`, Sutherland-law viscosity at `sim/USStandardAtmosphere.cpp:150`) — yet those two US Standard overrides have no callers at all, production or test; only the constant and vacuum overrides of the same interface are exercised. No Mach or Reynolds number is computed anywhere (`sim/VacuumAtmosphere.h:21` even warns hypothetical Mach callers to guard against a zero speed of sound), so C_d is speed-independent: a flight on an upper-class motor from the 1/4A–M ladder will pass Mach 1 under subsonic-constant drag without comment.

The pattern across this section is uniform and worth naming once: the consumed physics is small and trustworthy, and nearly every missing capability already has its seam built — value types, interface methods, recorded state, even correct formulas — with the final wiring left undone. That is a deliberate staging strategy, and the in-tree comments say so; but until the wiring lands, QtRocket’s aerodynamic fidelity is a constant- C_d point mass flying straight up, and it should be evaluated as exactly that.

7 User interfaces: GUI, CLI, and visualizer

QtRocket ships three executables over the one simulation core surveyed in [Section 2](#): the Qt6 Widgets application `qtrocket`, the headless REPL `qtrocket-cli`, and the standalone OpenGL viewer `qtrocket-visualizer`. Each links `qtrocket_core` and nothing of the other two; the visualizer’s build file states outright that it does not touch the GUI or CLI code (`visualizer/CMakeLists.txt:5`). This section shows the document’s recurring motif in its most visible form: capability accumulates in the core, and the front ends expose sharply uneven subsets of it. The CLI is the broadest surface — it can author, persist, and fly a multi-part design; the visualizer is the most finished piece of visual UI in the project; and the GUI, the executable that carries the project’s name, is the narrowest — a motor-selection-and-launch panel whose design-tool half is empty scaffolding at commit [254cd41](#).

7.1 The Qt6 GUI: a motor-and-launch front end

`main.cpp` instantiates the logger and the `QtRocket` controller singleton, then blocks in `gui::run` (`main.cpp:25`). That function — the only bridge between the Qt-free controller and Qt — spawns a `std::thread` running `guiWorker`, which constructs `QApplication`, installs a translator, creates `MainWindow`, and enters `exec()` (`gui/GuiRunner.cpp:30`), then immediately joins it.

QApplication on a spawned thread

`std::thread guiThread(guiWorker, ...); guiThread.join();` — the Qt event loop runs on a secondary thread while the main thread blocks in `join()` for the application’s lifetime (`gui/GuiRunner.cpp:59`). The indirection buys nothing — the caller does no concurrent work — while putting the GUI on a non-main thread, which Qt only tolerates on some platforms and which is a hard failure on macOS, where the GUI must run on the first thread. **FAIL**

Startup also performs two loads that can never succeed. The application icon is read from `:/qtrocket.png` (`gui/GuiRunner.cpp:31`), a path absent from the project’s only `.qrc`, whose sole entry is `resources/rocket64.png` under `/images` (`qtrocket.qrc:4`); the icon silently resolves to null, masked because the main window loads the valid path separately (`gui/MainWindow.ui:16`). And the internation-

alization scaffolding is dead code: the translator loads `qtrocket_<locale>` from a `:/i18n/` resource prefix that is never populated (`gui/GuiRunner.cpp:40`) — the build-side half of the story, translation files generated but attached to nothing, is covered in [Section 9](#) — and the adjacent `TODO` admits the application is `en_US` only (`gui/GuiRunner.cpp:34`).

The main window’s layout promises a design tool it does not contain. The `.ui` places a `RocketTreeView` (a promoted `QTreeView`), a tab widget, and a `RocketModelerView` in the central splitter (`gui/MainWindow.ui:49`); the constructor adds two tabs programmatically — `CannonballTab` and `SimOptionsTab` (`gui/MainWindow.cpp:30`) — and wires exactly three menu actions: Quit, Save Motor Database, and About (`gui/MainWindow.cpp:42`). Both design widgets are default-constructed shells with no model, no data, and no behavior; the complete implementation of the tree widget at `HEAD` is (`gui/RocketTreeView.cpp:1`):

```
#include "RocketTreeView.h"

RocketTreeView::RocketTreeView(QWidget* parent) : QTreeView(parent)
{
}
}
```

`RocketModelerView` is the same empty `QWidget` shell (`gui/RocketModelerView.cpp:3`), so the left panel and bottom panel render permanently blank. The File menu completes the picture: New, Open, Save, Close, and Save As are declared but ship `enabled=false` in the `.ui` — `actionNew` at `gui/MainWindow.ui:118`, Open, Save, Close, and Save As at lines 130, 142, 154, and 171 — with no slots at `HEAD`. Every one of these capabilities exists one layer down: the CLI’s `newdesign` (`cli/Repl.cpp:784`) and `addpart` (`cli/Repl.cpp:825`) author part trees the GUI cannot even display. Structural design status in the GUI at the reviewed commit: **Absent** (**WIP** on the in-flight branch of [Section 7.4](#)).

What the GUI does do is launch point masses, and the tab name is honest about it. The `CannonballTab` takes initial velocity, a launch angle validated to 0–90° from vertical (`gui/CannonballTab.cpp:36`), mass, drag coefficient, and reference area (`gui/CannonballTab.cpp:40`). The launch slot decomposes the velocity into downrange and vertical components (`gui/CannonballTab.cpp:104`), writes mass, C_d , and area into the shared `RocketModel` through the `QtRocket` facade — the GUI’s only route to the core (`gui/CannonballTab.cpp:110`) — and calls `launchRocket()` (`gui/CannonballTab.cpp:115`). That call is fully synchronous: it runs `runUntilTerminate()` on the calling thread (`QtRocket.cpp:54`), freezing the window for the duration of the propagation before `AnalysisWindow` opens (`gui/CannonballTab.cpp:118`). One rule gates the workflow: Calculate Trajectory is enabled iff the rocket has a motor (`gui/CannonballTab.cpp:88`), a condition reachable by four acquisition paths: (i) local file import of `RockSim .rse` or `RASP .eng`, dispatched by extension with exceptions caught into a critical dialog (`gui/CannonballTab.cpp:136`, `gui/CannonballTab.cpp:141`); (ii) a `.qmd` motor-database round trip with suffix enforcement (`gui/CannonballTab.cpp:173`, `gui/CannonballTab.cpp:212`), duplicated in the Tools menu (`gui/MainWindow.cpp:74`); (iii) online search through the `ThrustCurveMotorSelector` dialog (`gui/CannonballTab.cpp:163`); and (iv) an engine combo rebuilt from the database as the single source of truth (`gui/CannonballTab.cpp:156`) plus Set Motor, which resolves the name and installs the motor (`gui/CannonballTab.cpp:231`).

The online selector is a three-click manual workflow — “Get Metadata” populates search facets (`gui/ThrustCurveMotorSelector.cpp:52`), “Search” merges results into the local database (`gui/ThrustCurveMotorSelector.cpp:75`), “setMotor” installs the selection and plots its thrust curve (`gui/ThrustCurveMotorSelector.cpp:83`) — and every click performs network I/O synchronously on the GUI thread; the core documents the cost itself, noting downloads are “performed synchronously on the caller’s (often GUI) thread” (`model/ThrustCurveClient.cpp:292`), with searches capped at 100 results

(`model/ThrustCurveClient.cpp:295`). Failures are silent by construction: `CurlConnection` returns an empty string on any transport or HTTP failure (`utils/CurlConnection.h:28`), the client degrades to empty results with Logger-only errors (`model/ThrustCurveClient.cpp:271`), and the GUI slots have no failure branch (`gui/ThrustCurveMotorSelector.cpp:48`) — a user without connectivity clicks Get Metadata and simply sees nothing happen.

The one polished corner is `SimOptionsTab`: a live-apply form whose atmosphere, gravity, and integrator combos are enumerated from the sim layer’s own model registries via throwaway `Environment/Integrator` instances (`gui/SimOptionsTab.cpp:32`; registries at `sim/Environment.h:42`, `sim/Integrator.h:43`), so the GUI cannot drift from the core’s vocabulary. A one-time apply syncs `QtRocket` to the displayed defaults (`gui/SimOptionsTab.cpp:71`), and gravity/atmosphere changes mutate the shared `Environment` in place so the two selections do not clobber each other (`gui/SimOptionsTab.cpp:93`).

Present

Post-run analysis is thinner. `AnalysisWindow` embeds a vendored `QCustomPlot v2.1.1` (`gui/qcustomplot.h:23`) and offers exactly three graphs: altitude (`position[2]` vs. time, `gui/AnalysisWindow.cpp:49`), vertical velocity (`gui/AnalysisWindow.cpp:74`), and the motor thrust curve (`gui/AnalysisWindow.cpp:86`). The axis-range code dereferences `std::min_element` without an emptiness check (`gui/AnalysisWindow.cpp:111`; repeated at `gui/ThrustCurveMotorSelector.cpp:113`) — undefined behavior on an empty series, and an empty default `ThrustCurve` does exist (`model/ThrustCurve.h:17`); only the motor-gated launch flow keeps the path unreachable. More telling is what the window never shows: the controller exposes whole-flight `TrajectoryStatistics` (`QtRocket.h:55`) and a `TerminationReason` (`QtRocket.h:62`), and the propagator records mass, CG, and inertia alongside every state — all of which the CLI surfaces (`cli/Repl.cpp:703`, `cli/Repl.cpp:104`) — yet neither accessor has a single call site under `gui/`. A user whose run aborted for no-liftoff gets an altitude plot and no explanation, and must eyeball the curve for apogee. The Barrowman center-of-pressure machinery of [Section 6](#) is likewise computed and tested in the core with no display surface in any front end. **Partial**

The remaining gaps are individually small and collectively diagnostic: no `QSettings` or other preference persistence exists anywhere in `gui/`, `visualizer/`, or `main.cpp` — file dialogs hardcode `/home` as their start directory (`gui/CannonballTab.cpp:127`, `gui/MainWindow.cpp:78`) — dialogs keep the Designer default title “Dialog” (`gui/AnalysisWindow.ui:14`), and `AnalysisWindow.ui` declares a `plotAltitude()` slot that does not exist in the class (`gui/AnalysisWindow.ui:77`). None is hard to fix; together they mark a front end that has not yet had a first polish pass.

7.2 The command-line REPL

`qtrocket-cli` is the inverse of the GUI: no widgets, no polish debt, and the widest command surface in the project. Its main deliberately ignores `argc/argv` (`cli/CliMain.cpp:13`) — no flags, no script-file argument, no `-version` — and sets the logger to `ERROR` because “Logger shares stdout with the CLI’s machine-readable output” (`cli/CliMain.cpp:15`), then runs `cli::Repl` over `std::cin/std::cout` (`cli/CliMain.cpp:25`). The loop is a bare `std::getline` with a flush per command (`cli/Repl.cpp:232`); there is no readline, no history, and no prompt — the project’s own `CLAUDE.md` calls it a “readline-style REPL,” but no such library is linked (`CMakeLists.txt:297`). That austerity is the documented intent: the class comment promises the same operations the GUI performs, “interactively, from a pipe, or from a redirected script” (`cli/Repl.h:20`), and batch use is plain shell redirection.

The stdout protocol is the interface contract

Every command answers exactly one OK or ERR line (dispatch chain at `cli/Repl.cpp:245`, unknown-command fallback at `cli/Repl.cpp:981`); `launch` appends WARN: on non-nominal termination — through an exhaustive switch with no default, so a new `TerminationReason` fails to compile until the CLI handles it (`cli/Repl.cpp:41`) — and a CSV `<abs-path>` line; informational output is #-prefixed and # lines are comments on input (`cli/Repl.cpp:256`). Because the REPL is built as a static `cli` library (`CMakeLists.txt:235`), the test binaries drive the exact object code users type into and parse this same protocol (Section 9).

Table 4 inventories the command set. The count deserves precision: the dispatch chain has 32 handler branches covering 33 command words (`quit` and `exit` share one). The grammar for design authoring is `key=value` over the part factory (`cli/Repl.cpp:169`; `model/parts/PartFactory.cpp:27`), with strict full-string numeric parsing that rejects trailing garbage (`cli/Repl.cpp:119`) and warn-and-skip tolerance for unknown keys (`cli/Repl.cpp:205`).

Table 4. The `qtrocket-cli` command set at commit 254cd41: 33 command words, 32 handler branches in `Repl::execute` (`cli/Repl.cpp:245`).

Command	Behavior
<i>Session</i>	
<code>help</code>	command list, #-prefixed
<code>quit / exit</code>	“OK bye”; ends the loop
<i>Motor database</i>	
<code>loadmotors <file></code>	import <code>RockSim .rse / RASP .eng</code> , by extension
<code>savedb / loaddb <file.qmd></code>	motor-database XML save; additive load
<code>listmotors [substr]</code>	filtered summaries (avg thrust, total impulse)
<code>setmotor <code></code>	resolve by common name, install on the rocket
<code>tcfacets</code>	thrustcurve.org search facets (online)
<code>tcsearch <k=v> . . .</code>	online search (manufacturer, diameter, impulse class); merges into the database
<i>Simulation configuration</i>	
<code>setmass / setdrag / setarea</code>	written through to the <code>RocketModel</code> immediately
<code>setvelocity / setangle</code>	staged; applied at launch
<code>setimestep <s></code>	integrator step, > 0
<code>listatmospheres / setatmosphere</code>	names validated against the sim registries; likewise <code>listgravity/setgravity</code> and <code>listintegrators/setintegrator</code>
<code>status</code>	staged configuration plus database size
<i>Design authoring</i>	
<code>newdesign <type> [k=v. . .]</code>	root part via the <code>makePart</code> factory
<code>addpart <parent root> <type> [k=v. . .]</code>	attach with <code>abut(z)</code> ; <code>x/y</code> parsed but ignored
<code>listparts</code>	indented id/type/name/mass tree; composite mass/CG footer
<code>removepart <id></code>	refuses the root; re-checks motor presence
<code>cleardesign</code>	restore the boot placeholder
<i>Flight and persistence</i>	
<code>launch</code>	run to termination; stats, WARN on abort, auto-CSV
<code>states [stride]</code>	state series as CSV to stdout
<code>save <path.csv></code>	full series to file; echoes the absolute path
<code>savedesign / loaddesign <file.qrd></code>	design round trip (Section 8)

This inventory is not a paper capability; the review exercised it end to end against the built binary. A single piped session loaded 249 motors from the bundled `Aerotech.rse` (`cli/Repl.cpp:314`), loaded

the committed `large38_multi.qrd` fixture with its motor re-resolved by name (“OK loaddesign: ... (motor I280DM)”), listed the part tree with its composite mass/CG footer (`cli/Repl.cpp:880`), and flew it: OK launch: 4258 steps, `t_final=42.570s`, apogee 1856.613 m at $t = 13.240$ s, maximum speed 562.928 m/s at $t = 1.730$ s, statistics read from the live-tracked `TrajectoryStatistics` rather than re-derived (`cli/Repl.cpp:703`). A second fixture under the adaptive Runge-Kutta-Fehlberg integrator completed in 440 steps, an order of magnitude fewer than the fixed-step run, and a CLI-built design round-tripped through `savedesign/cleardesign/loaddesign` with identical part masses (Section 8 covers the mechanics).

The REPL’s gaps are edges, not holes. The process exit code is always zero (`cli/Repl.cpp:242`; the contract comment says so, `cli/Repl.h:28`), so the scripting mode that is the CLI’s stated strength cannot signal failure to a shell — callers must parse `ERR` lines. Every `launch` unconditionally writes `qtrocket_run.csv` into the current directory under a hardcoded name (`cli/Repl.cpp:716`). `addpart` parses `x=` and `y=` into an offset vector (`cli/Repl.cpp:189`) and silently discards them, attaching with `abut(offset.z())` only (`cli/Repl.cpp:872`) — and that `abut` is the only seat kind reachable from any front end, although the `.qrd` format round-trips all three (Section 8). No command sets the design name, so although the serializer writes and restores it (`model/DesignSerializer.cpp:221`), every CLI-authored file carries `<design name=""/>` (`cli/Repl.cpp:259`). `status` prints a staged `dry_mass` that `loaddesign` never re-syncs (`cli/Repl.cpp:966`): after loading a 0.785 kg design, the live session still reported “dry_mass = 1 kg” (`cli/Repl.cpp:656`) — harmless to the physics, which uses the composite tree mass, but misleading in the one place a user checks configuration. Two smaller notes: `DEG_PER_RAD` is a truncated copy of the GUI’s constant (57.2958 vs. $180/\pi = 57.29578\dots$) whose comment points at the wrong file (`cli/Repl.cpp:35`); and `setangle` tilts only the initial velocity vector (`cli/Repl.cpp:679`) — thrust remains world-vertical throughout the burn, a 3-DOF core limitation owned by Section 5.

7.3 The standalone visualizer

`qtrocket-visualizer` is built unconditionally from the root build (`CMakeLists.txt:243`) and links only the Qt OpenGL modules plus `qtrocket_core` (`visualizer/CMakeLists.txt:27`). Its main disables the RHI widget backing store and deliberately leaves the GL version and profile unconstrained — the documented rationale is surviving EGL/Wayland/NVIDIA context matching (`visualizer/main.cpp:10`); an optional `.qrd` path on `argv` is opened at startup (`visualizer/main.cpp:35`). The window owns a headless `RocketModel` and a deliberately empty `MotorModelDatabase` purely to satisfy the loader’s signature (`visualizer/VisualizerWindow.h:69`) — a missing motor is a harmless warning, since only geometry matters here — and opens designs through the same `model::DesignSerializer::load` the CLI uses (`visualizer/VisualizerWindow.cpp:184`), keeping the current model on failure (`visualizer/VisualizerWindow.cpp:202`).

The architectural keystone is where the meshes get their positions (`visualizer/RocketMesh.cpp:429`):

```
// Consume the same absolute placement the simulator does: resolve every part's pose once, then
// translate each origin-centered primitive so its fore plane lands at the resolved fore-plane
// origin (the build* primitives are centered about mid-length, half an axialLength forward of
// the aft plane).
const std::vector<model::part::Placed> placed =
    model::part::resolvePlacements(*root, model::part::Pose{});
```

One resolver, two consumers

`buildRocketMeshes` positions every primitive from `model::part::resolvePlacements` — the same routine the simulator’s composite mass/CM/inertia cache consumes (`visualizer/RocketMesh.cpp:433`; `model/parts/Part.cpp:166`) — so rendered geometry and simulated geometry

cannot diverge by construction. This closes, for the visualizer, exactly the silent renderer-vs-simulator divergence that motivated the placement redesign of [Section 3](#).

The same walk reads the cached overlap-diagnostics verdict from the part tree (`visualizer/RocketMesh.cpp:440`; `model/parts/Part.h:128`) and tags offending parts, which the GL widget paints in a fixed error red overriding any color scheme (`visualizer/RocketGLWidget.cpp:621`) — the physics layer’s self-intersection check made visible. Characteristically for this codebase, the diagnostic message is carried on each render item “for a tooltip / status line” that no UI consumes yet (`visualizer/RocketMesh.h:53`).

Mesh coverage is dispatched by `dynamic_cast` over the concrete part types (`visualizer/RocketMesh.cpp:456`): a cone for `ConicalNoseCone`, hollow walls with annular caps for `BodyTube` (`visualizer/RocketMesh.cpp:153`), flat-plate trapezoidal prisms arrayed around the body for `FinSet` (`visualizer/RocketMesh.cpp:335`), and a UV sphere for `HollowSphere`; `Motor` and unknown types get no geometry (`visualizer/RocketMesh.cpp:478`), so a loaded design’s motor is invisible. Primitives are origin-centered with +z toward the nose (`visualizer/RocketMesh.h:70`) and translated so the fore plane lands at the resolved origin (`visualizer/RocketMesh.cpp:482`). The rendering layer is genuinely portable: dual shader dialects (GLSL 1.20 desktop, GLSL ES 1.00) chosen via `context()->isOpenGLES()` (`visualizer/RocketGLWidget.cpp:314`), wireframe as a pre-expanded `GL_LINES` edge buffer because `glPolygonMode` is unavailable on GLES2 (`visualizer/RocketGLWidget.cpp:518`), and an orbit/pan/zoom camera with a bounding-sphere reset (`visualizer/RocketGLWidget.cpp:434`, `visualizer/RocketGLWidget.cpp:261`). Four editable color-scheme presets map `Part::typeName()` to colors (`visualizer/ColorScheme.cpp:22`), but the per-type swatch panel is hardcoded to exactly the four current type names (`visualizer/VisualizerWindow.cpp:44`); a fifth part type would render in the fallback color with no picker.

The one thing the visualizer lacks is a caller. It is not integrated with the GUI in any way — the GUI never launches or embeds it (`gui/MainWindow.cpp:22`) — and at HEAD the GUI cannot even produce the `.qrd` files it consumes; only the CLI’s `savedesign` can. The most finished visual component in the project is unreachable from the product’s main window. **Present** as a tool, **Absent** as an integrated feature.

7.4 Work in flight: the `RocketTreeViewImplementation` branch

Everything above describes commit `254cd41`. An in-flight branch, `RocketTreeViewImplementation`, sits two commits ahead (`049bb02` “Initial implementation of RocketTreeView Widget”, `2ddd553` “GUI save/load WIP”; +360/−15 lines across ten files) and closes part of the GUI’s design gap. It adds `RocketPartModel`, a read-only `QAbstractItemModel` over the part tree with Name/Type/Mass columns, using raw `Part*` internal pointers and a new `Part::getParent()` accessor for the parent walk (declared at `model/parts/Part.h:212` on that branch). The tree stays live through a deliberately Qt-free seam: `RocketModel::setStructureChangedCallback`, a `std::function` observer hook added at `model/RocketModel.h:121` on the branch and fired from `setMotorModel`, `setRoot`, `addPart`, and `removePart`; the view re-roots and expands on every fire and detaches in its destructor. The branch also wires File > Open/Save/Save As to `DesignSerializer` for `.qrd` files with current-file tracking and window-title updates (handlers in the region of `gui/MainWindow.cpp:92` on the branch), and adds `CannonballTab::refreshFromModel()` so a load re-syncs the mass/ C_d /area fields rather than letting the next Calculate silently overwrite loaded values. Even on the branch the tree is display-only — no part can be added, removed, or edited from the GUI; New and Close remain disabled; and the refresh is a full model reset per change. The second commit’s own title, “GUI save/load WIP,” states the status accurately. **WIP**

One cross-cutting fact frames all three front ends: nothing under `gui/` or `visualizer/` has any automated test (Section 9), so every “implemented” verdict in this section rests on code reading and the live CLI sessions above, not on a suite. And the CLI’s class comment — “the same operations the GUI performs ... as text commands” (`cli/Repl.h:20`) — was written when the GUI was the reference point. At the reviewed commit the relationship has inverted: the sentence now undersells a REPL that authors, persists, and flies designs the GUI cannot yet open. The consolidated ranking of these gaps belongs to Section 11.

8 Persistence & interoperability

QtRocket persists three kinds of artifact: rocket designs (`.qrd`), motor databases (`.qmd`), and flight trajectories (CSV). All design persistence flows through one model-layer class, `model::DesignSerializer` — two static functions, `save` and `load`, over Boost.property_tree XML, deliberately mirroring the motor database’s serialization idiom (`model/DesignSerializer.h:31`, `model/DesignSerializer.h:16`). At this commit the serializer has exactly two consumers: the CLI’s `savedesign/loaddesign` commands (`cli/Repl.cpp:926`) and the standalone visualizer, which loads a `.qrd` passed on its command line (`visualizer/VisualizerWindow.cpp:184`). The GUI does not call it at all — a `grep` under `gui/` finds no use — which repeats the pattern this review keeps meeting: the persistence seam is finished and heavily tested before its most obvious client exists (Section 7).

8.1 The `.qrd` design format

A `.qrd` file is versioned XML: a `QtRocketDesign` root stamped `version="0.2"` (`model/DesignSerializer.cpp:220`) with a design name (`model/DesignSerializer.cpp:221`), a recursive polymorphic part tree, an optional `<motor>` element, and a `<sim>` block. Listing 8 shows a complete committed fixture.

Each `<part>` carries its factory `type` and `name`; its `<params>` come from the factory reflector `part::params(node)`, which reads geometry fields only (`model/DesignSerializer.cpp:101`, `model/parts/PartFactory.cpp:79`). Siblings survive because children are appended with `add_child`, not `put` (`model/DesignSerializer.cpp:115`) — a property_tree trap pinned by a dedicated round-trip test (`tests/DesignPersistenceTests.cpp:57`). Placement is a `<link>` element carrying the `StationLink` fields — seat kind (`Abut`, `NestInBore`, `OnSurface`), fractional parent and child stations in $[0, 1]$, and a gap in metres — with the seat enum round-tripped through one shared string table (`model/DesignSerializer.cpp:73`). A default-equal link (the zero-config `abut`) is elided on write and recovered as the default on read (`model/DesignSerializer.cpp:104`, `model/DesignSerializer.cpp:86`); the reserved 6-DOF `childRot` quaternion is deliberately not persisted while it is identity by construction (`model/DesignSerializer.cpp:85`). The `<sim>` block carries exactly the three RocketModel-owned aero scalars — `dragCoefficient`, `referenceArea`, `referenceAreaOverridden` (`model/DesignSerializer.cpp:233`); launch and environment options (velocity, angle, atmosphere, gravity, integrator) are session state by declared policy and are never written (`model/DesignSerializer.cpp:230`).

Intent in, geometry out

The file stores placement *intent* (seat, stations, gap) and never a resolved pose; the single placement resolver re-derives absolute geometry on every load (`model/DesignSerializer.cpp:93`). Serialization therefore cannot disagree with the simulator or the visualizer about where a part sits — the divergence that killed the legacy 0.1 format (Section 3) is unrepresentable in 0.2.


```

<?xml version="1.0" encoding="utf-8"?>
<QtRocketDesign version="0.2">
  <design name=""/>
  <part type="NoseCone" name="Nose">
    <params baseRadius="0.02" length="0.12" wallThickness="0" density="1400" solid="true"/>
    <children>
      <part type="BodyTube" name="Airframe">
        <params innerRadius="0.019099999999999999" outerRadius="0.02" length="0.40000000000000002"
          ↪ density="1400"/>
        <link seat="Abut" parentStation="0" childStation="1" gap="-0.029999999999999999"/>
        <children>
          <part type="FinSet" name="Fins">
            <params density="1400" rootChord="0.070000000000000007" tipChord="0.029999999999999999"
              ↪ span="0.050000000000000003" sweep="0.040000000000000001" thickness="0.003000000000000001"
              ↪ bodyRadius="0.02" finCount="3"/>
            <link seat="OnSurface" parentStation="0" childStation="0" gap="-0.008666666666666659"/>
            <children/>
          </part>
        </children>
      </part>
    </children>
  </part>
</children>
<sim dragCoefficient="1" referenceArea="0.001134" referenceAreaOverridden="false"/>
</QtRocketDesign>

```

Listing 8. A complete .qrd design file (tests/data/designs/large38_basic.qrd). Doubles are written at full precision, and placement is stored as <link> intent, never as a resolved pose.

8.2 Round-trip fidelity

What round-trips exactly, verified by fourteen serializer-level tests in tests/DesignPersistenceTests.cpp plus a fixture-wide sweep (tests/DesignMatrixTests.cpp:417): geometry parameters and tree structure, non-default station links with all three seat kinds (tests/DesignPersistenceTests.cpp:380), motor identity with its thrust and mass curves (tests/DesignPersistenceTests.cpp:138), and the drag coefficient with the reference-area override flag (tests/DesignPersistenceTests.cpp:100). What does not survive is more instructive:

Per-part mass overrides. setmass writes the root part’s own mass (model/RocketModel.h:87), but <params> reflect only the factory geometry fields — PartParams has no mass field — so the override is silently absent from the written file and a reload recomputes mass from geometry (model/parts/PartFactory.cpp:79, model/DesignSerializer.cpp:31).

Motor placement. writePart skips the Motor node in the tree (model/DesignSerializer.cpp:114); only the common name is stored (model/DesignSerializer.cpp:227). On load, setMotorModel re-derives a CM-station abut link with gap motorOffset.z() (model/RocketModel.cpp:124) — loss-less today only because that offset is documented “Zero today” (model/RocketModel.h:130). The moment motor placement becomes user-settable, it will not round-trip.

Part identity. Ids are process-unique and never serialized; a reload reassigns them, as the CLI help itself warns (cli/Repl.cpp:294).

Design name, from the CLI. The serializer writes and restores design@name (model/DesignSerializer.cpp:221, model/DesignSerializer.cpp:259) and RocketModel has the accessors (model/RocketModel.h:65), but no REPL command sets it, so every CLI-authored file carries name="" — visible in Listing 8.

Session configuration. Deliberate, per the policy above: a `.qrd` describes a rocket, not an experiment. Reproducing a specific flight requires re-staging integrator, atmosphere, and launch settings by hand or script.

setmass survives the session, not the file

Verified live against the built binary: `setmass 5` followed by `savedesign` wrote `<params>` containing only `innerRadius/outerRadius/density` — no mass attribute — and the reloaded design reverted to its geometry-derived mass. No warning is emitted at save time. **FAIL**

Motor faithfulness is database-relative

The motor is re-resolved by common name against whatever database the loading session supplies (`model/DesignSerializer.cpp:271`). A miss logs a warning and loads the geometry without a motor — never throws (`tests/DesignPersistenceTests.cpp:167`). A `.qrd` therefore reproduces its author’s rocket only when paired with an equivalent motor database; the file itself carries no curve data.

One persisted field deserves a flag: `referenceAreaOverridden` round-trips faithfully (`model/DesignSerializer.cpp:235`), but the non-overridden case is supposed to fall back to a geometry-derived reference area, and that derivation is never invoked on any production path — a loaded design without an override flies on the default $1.134 \times 10^{-3} \text{ m}^2$ scalar regardless of its geometry (Section 6). The format reserved the semantics before the consumer was wired.

8.3 Loader error handling and versioning

The reader is fail-safe in structure and fail-closed in vocabulary. The whole geometry tree is built detached and installed atomically with one `setRoot` call, so a malformed file, unknown part type, bad seat, or failed attach leaves the current rocket untouched (`model/DesignSerializer.cpp:255`); because `addChildPart` is a silent no-op on rejection, attach success is verified by a child-count delta that converts silence into a throw (`model/DesignSerializer.cpp:204`). An unknown seat kind is rejected outright (`model/DesignSerializer.cpp:184`, pinned at `tests/DesignPersistenceTests.cpp:355`), as is a `Motor` masquerading as a tree part (`model/DesignSerializer.cpp:157`). Versioning is a major-version gate: any minor under major 0 is accepted, anything else throws before the rocket is touched (`model/DesignSerializer.cpp:250`, `tests/DesignPersistenceTests.cpp:189`). There is no migration machinery at all: a legacy 0.1 file — recognizable by its CM-to-CM `<offset>` element — is rejected with an explicit “must be re-created” error rather than silently mis-placed (`model/DesignSerializer.cpp:194`, `tests/DesignPersistenceTests.cpp:333`). For a format this young, rejection is a defensible substitute for migration; it will not stay defensible once users accumulate files.

8.4 The placement-invariance baseline

The migration from CM-to-CM offsets to station links was made falsifiable by a frozen physics snapshot: `tests/data/placement-invariance-baseline.txt` declares itself “IMMUTABLE GROUND TRUTH, generated from the LEGACY CM-to-CM reader BEFORE the placement refactor” (`tests/data/placement-invariance-baseline.txt:3`). It is a `format 1` text file (`tests/data/placement-invariance-baseline.txt:20`) recording, per design, composite mass, CM, the row-major inertia tensor about the CG, CP with a validity flag, and per-part DFS stations, all printed `%.17g` for exact IEEE-754 round-trip (`tests/data/placement-invariance-baseline.txt:18`), taken with an empty motor database so the snapshot is the pure airframe composite. The corpus is fixed at 24 `.qrd` fixtures spanning seven diameter classes from `micro13` to `xl75` (`tests/PlacementInvarianceTests.cpp:64`;

the 25th committed fixture, `tests/data/designs/xl75_multi_pokethrough.qrd`, is deliberately excluded because it must fail the overlap gate). The consuming tests re-snapshot every fixture through the current reader and assert mass and inertia identical to 10^{-9} relative (`tests/PlacementInvarianceTests.cpp:266`), CG and every part station identical after a single tip-datum shift (`tests/PlacementInvarianceTests.cpp:286`, `tests/PlacementInvarianceTests.cpp:299`), a worst-drift diagnostic (`tests/PlacementInvarianceTests.cpp:328`), and save→reload idempotence through the 0.2 writer (`tests/PlacementInvarianceTests.cpp:358`). CP is deliberately not compared: the legacy CP was CM-contaminated, and correcting it was the point. Two hand-pinned CM values at 10^{-12} guard sign and datum slips the aggregate gate could miss (`tests/PlacementInvarianceTests.cpp:445`).

One caveat: both the baseline header and the test file describe regeneration as “gated behind the `QTROCKET_REGEN_PLACEMENT_BASELINE` env var” (`tests/data/placement-invariance-baseline.txt:6`, `tests/PlacementInvarianceTests.cpp:31`), but no code anywhere in the tree reads that variable — the bootstrap writer was evidently deleted after use. The immutability contract is thereby enforced rather more literally than intended: regenerating the baseline today would mean re-writing the tool.

Partial

8.5 Motor database files (.qmd)

The motor database persists to its own XML format, owned by [Section 4](#); the interop-relevant facts are these. The writer stamps `QtRocketMotorDatabase version="0.1"` (`model/MotorModelDatabase.cpp:184`) and emits one `<motor>` node per entry with every metadata field, delays as a CSV string, and the thrust curve as `<thrust time= force= />` samples; the loader applies per-field defaults, skips foreign nodes, and recomputes the derived mass curve — but never checks the version attribute it wrote (`model/MotorModelDatabase.cpp:249`), so this format has no migration hook either. The round trip over the full bundled AeroTech import (252 `<engine>` entries collapsing to 249 name-keyed motors) is exact (`tests/MotorDatabasePersistenceTests.cpp:297`), and a committed fixture (`tests/data/motors/estes_small.qmd`) uses `.qmd` to pin online-sourced motors offline for the unit-conversion regression tests.

8.6 Import and export surface

Format	Direction	Status	Notes
.qrd 0.2 (native design)	read/write	Present	CLI and visualizer; GUI not wired at this commit
.qrd 0.1 (legacy design)	—	Absent	rejected fail-closed, no migration (<code>model/DesignSerializer.cpp:194</code>)
.qmd (native motor DB)	read/write	Present	version written, never checked (<code>model/MotorModelDatabase.cpp:249</code>)
RockSim <code>.rse</code> (motors)	import	Present	per-sample mass/CG data dropped (Section 4)
RASP <code>.eng</code> (motors)	import	Present	strict validating parser; extension-dispatched (<code>cli/Repl.cpp:314</code>)
thrustcurve.org REST (motors)	online import	Present	search facets + per-motor download (Section 4)
Flight trajectory CSV	export	Present	fixed 14-column schema (<code>cli/Repl.cpp:104</code>)
OpenRocket <code>.ork</code> (designs)	—	Absent	no read or write
RockSim <code>.rkt</code> (designs)	—	Absent	motor import only, no design import
CSV import (any)	—	Absent	

Table 5. File-format interoperability at commit `254cd41`.

[Table 5](#) summarizes the surface. The only export beyond the native formats is the flight CSV,

one shared writer with the fixed header `t,x,y,z,vx,vy,vz,mass,cg_x,cg_y,cg_z,Ixx,Iyy,Izz` at 9 significant digits (`cli/Repl.cpp:104`) — time, position, velocity, and the time-varying composite mass, CG, and inertia diagonal the propagator records. It is emitted three ways: unconditionally to a hardcoded `qtrocket_run.csv` in the working directory on every launch (`cli/Repl.cpp:716`), on demand to stdout with a stride (`cli/Repl.cpp:754`), and to a caller-named file (`cli/Repl.cpp:781`). There is no column or unit selection.

On the import side the asymmetry is deliberate and worth stating plainly: QtRocket imports foreign *motor* data through three independent paths — matching OpenRocket’s motor-database reach, including thrustcurve.org — but imports no foreign *design* format and exports to none. There is no `.ork` reader or writer, no RockSim `.rkt` design import, and no CSV ingestion of any kind; designs enter the ecosystem only by being authored in QtRocket, and its own 0.1 files are locked out. Combined with the five-type part catalog (Section 3), this makes `.qrd` a closed world at this commit; the gap against OpenRocket’s zipped-XML `.ork` — which carries ~20+ component types, per-component overrides, flight configurations, and embedded simulations, with loaders back to version 1.0 — is quantified in Section 10, and the consolidated remediation list lives in Section 11.

9 Testing, coverage, and engineering infrastructure

The engineering infrastructure is the part of QtRocket that least resembles a young project: a single CMake project pinned to C++23 (`CMakeLists.txt:5`), every dependency except Qt6 vendored at an exact tag, warnings-as-errors on four compiler families, 239 test cases checked largely against independent oracles rather than golden values, a five-platform CI matrix, and branch-level source coverage wired into the build. The same inspection finds the document’s recurring motif in infrastructural form — machinery finished to a high standard with no consumer: the coverage target is run by no CI job, the translation rules produce files no target references, the install rule ships one of three executables, and the project version surfaces nowhere a user could see it. This section reviews both halves.

9.1 Build system: presets, pinned dependencies, strict warnings

`CMakePresets.json` defines eight configure presets — `debug`, `release`, `debug-clang`, `release-clang`, `debug-gcc`, `release-gcc`, `release-msvc`, and `coverage-clang` — every one pinned to the Ninja generator (`CMakePresets.json:7`) and paired with matching build and test presets. MSVC exists only as a release variant (`CMakePresets.json:70`); there is no `debug-msvc`. The coverage preset inherits `debug-clang` but redirects to a separate `build-coverage/` tree (`CMakePresets.json:35`) and sets `QTROCKET_ENABLE_COVERAGE` (`CMakePresets.json:39`), so instrumented builds never disturb the normal build directory.

All non-Qt dependencies arrive via FetchContent at exact tags, enumerated in Table 1; Qt6 is the only system dependency (`CMakeLists.txt:140`), which makes the Qt5/Android fallback branch at `CMakeLists.txt:202` unreachable dead code — a safe deletion candidate.

Hermetic dependency closure

`curl` is built as a static, `HTTP_ONLY` library (`CMakeLists.txt:47`) with every optional third-party backend — `libssh2`, `nghttp2`, `libidn2`, `libpsl`, `brotli`, `zstd` — forced off. The in-tree comment records why (`CMakeLists.txt:51`): several of these knobs default to ON/AUTO, so `curl` links whatever it auto-detects on the build host, and auto-detected Homebrew libraries on the macOS CI runner previously broke the final link. The forced-off block converts an environment-

dependent link into a reproducible one.

One knob in that block is inert: `set(SSL_ENABLED ON)` at `CMakeLists.txt:48` is not a curl cache option at all — the real switch, `CURL_ENABLE_SSL`, defaults ON in the fetched source (`build/_deps/curl-src/CMakeLists.txt:725`). HTTPS works, but through curl’s defaults, not through line 48; the line misleads a reader about how TLS is enabled.

Warning hygiene is strict tree-wide: `-Wall -Wextra -Wpedantic -Werror` on GCC/Clang (`CMakeLists.txt:97`), `/W4 /WX` on MSVC (`CMakeLists.txt:88`), applied after the `FetchContent` blocks so vendored code is exempt — with one deliberate exception: the vendored `gui/qcustomplot.cpp` is compiled `-w` (`CMakeLists.txt:154`) so `-Werror` can stay on for project code. Against this, a comment in the same block claims “We also run ASAN and clang-tidy for additional security” (`CMakeLists.txt:93`); nothing backs it — no `.clang-tidy` file, no `-fsanitize` flag, no lint preset, no CI job. Sanitizers and static analysis are **Absent**, and the comment is stale. The layered link graph the build enforces (`utils` → `sim` → `model` → the `qrocket_core` static library at `CMakeLists.txt:224`, shared by every executable) is covered in [Section 2](#); the bottom-layer invariant is stated in the build itself (`utils/CMakeLists.txt:13`) and holds — nothing in `utils/` includes a `model/` or `sim/` header.

9.2 The test suite: six binaries, 239 cases

Testing is GoogleTest throughout: six binaries totalling 239 `TEST/TEST_F` cases, summarized in [Table 6](#).

Binary	Cases	Focus
<code>model_tests</code>	144	Composite-tree mechanics (dirty flags, parallel-axis shifts, deep clone); part geometry against closed-form, numeric, and mesh oracles; Barrowman aero composition; motor burn; thrust-curve interpolation; JSON/RASP motor parsing; placement resolver and diagnostics
<code>sim_tests</code>	27	US Standard Atmosphere tables; pluggable environment selection; RKF45 against closed-form dynamics; spherical gravity and geoid; integrator-selection guards
<code>integration_tests</code>	43	End-to-end physics through the controller (11); motor-database persistence (9); <code>.qrd</code> persistence (14); placement invariance against a committed baseline (7); poke-through regressions (2)
<code>propagator_tests</code>	6	Termination guards driven by a mock <code>Propagatable</code> : no-liftoff, non-finite state, max-sim-time backstop, clean abort on integrator exception
<code>cli_tests</code>	3	Scripted REPL build-save-reload-fly; <code>key=value</code> validation; RASP motor loading
<code>design_matrix_tests</code>	16	25-fixture corpus; mass scaling laws; 1/4A→M flight ladder (apogee monotonic in impulse); drag and integrator cross-checks
total	239	

Table 6. The six GoogleTest binaries and their coverage map. Case counts by `grep` of `TEST/TEST_F` at the pinned commit.

The suite’s character is the project’s strongest engineering asset: physics is validated against independent oracles — closed-form inertia formulas, numeric disk integration, mesh cross-checks ([Section 3](#)) — rather than recorded outputs; persistence is locked by round-trips plus a committed bit-identical invariance baseline over the fixture corpus ([Section 8](#)); and each of the propagator’s historical failure modes has a dedicated regression ([Section 5](#)). Motor data is tested at three altitudes: the `thrustcurve.org` JSON parsers are pure functions fed canned JSON string

literals, no network involved (`model/tests/ThrustCurveParserTests.cpp:1`); the database's online-source merge is driven through a `FakeThrustCurveAPI` implementing the `model::ThrustCurveAPI` seam (`tests/MotorDatabasePersistenceTests.cpp:106`); and the design-matrix suite flies real motors end to end. Fixture location is wired by absolute-path compile definitions (`QTROCKET_DATA_DIR`, `QTROCKET_TEST_DATA_DIR`, `tests/CMakeLists.txt:15`), so every binary runs from any working directory.

Registration with `ctest` follows one pattern everywhere: each binary is registered per-case via `gtest_discover_tests` and again as a single aggregate `qtrocket_*` entry (`model/tests/CMakeLists.txt:29`; `tests/CMakeLists.txt:25`), giving seven aggregate entries in total. The design-matrix binary carries the seventh — a third registration of the same executable, `qtrocket_design_matrix_heavy_tests`, which reruns only the `FlightMatrix/Atmosphere` sweeps under the `ctest` label `heavy` with `QTROCKET_FULL_LADDER=1` in its environment (`tests/CMakeLists.txt:96`). That environment variable is the whole mechanism: the flight sweeps ask `motorsToFly()` which motors to fly, and it returns either one motor per impulse class or the class's complete ladder (`tests/DesignMatrixTests.cpp:375`):

```
std::vector<std::string> motorsToFly(const Ladder& lad)
{
    if(std::getenv("QTROCKET_FULL_LADDER") != nullptr)
        return lad.motors;
    return { lad.motors.front() };
}
```

One binary, three ctest entries

The double registration means an unfiltered `ctest` runs every case twice — once discovered, once inside its aggregate. The documented workflows always filter (`-R 'qtrocket_*` for everything, `-LE heavy` for the fast loop, `-L heavy` for the sweeps alone, `tests/CMakeLists.txt:85`), so this is a footgun rather than a bug — but one a newcomer running bare `ctest` will step on. The heavy/fast split itself is well designed: one binary, an env-var gate, and a `ctest` label buy a fast default loop and an on-demand full-ladder sweep with no code duplication.

9.3 Coverage tooling

Coverage is Clang/LLVM source-based and gated hard: `QTROCKET_ENABLE_COVERAGE` fatally errors on any non-Clang compiler (`CMakeLists.txt:101`), then applies `-O0 -g -fprofile-instr-generate -fcoverage-mapping` (`CMakeLists.txt:108`). A custom coverage target shells out to `cmake/RunLlvmCoverage.cmake`, which runs `ctest` under `LLVM_PROFILE_FILE` (`cmake/RunLlvmCoverage.cmake:33`), aborts on any test failure (`cmake/RunLlvmCoverage.cmake:39`), merges profiles with `llvm-profdata`, and emits both a text summary and a branch-annotated, demangled HTML report (`cmake/RunLlvmCoverage.cmake:99`). This is genuinely good tooling — and no CI job runs it, so coverage numbers exist only on a developer's machine. Verdict: **Partial**, for that reason and for a real defect in its binary list.

Coverage target out of sync with the design-matrix suite

`QTROCKET_COVERAGE_BINARIES` (`CMakeLists.txt:248`) and the target's `DEPENDS` (`CMakeLists.txt:267`) name five test binaries — `design_matrix_tests` is missing from both — while the `ctest` regex the script is handed, `^qtrocket_.*tests$` (`CMakeLists.txt:261`), matches both design-matrix entries. On a fresh `build-coverage` tree, `-target coverage` therefore builds only the five listed binaries, `ctest` then tries to run the design-matrix entries whose executable was never built, and the script aborts with `FATAL_ERROR` (`cmake/RunLlvmCoverage.cmake:39`). Even when the binary happens to

exist, the heavy full-ladder sweep runs under instrumentation, and functions unique to that suite are never attributed in the report. **FAIL**

9.4 Continuous integration

CI is a GitHub Actions workflow triggered on push and pull request to `development` only (`.github/workflows/cmake-multi-platform.yml:13`). A four-job matrix with fail-fast off — Linux/GCC, Linux/Clang, macOS/AppleClang, and Windows/MSVC, each driving its release preset (`.github/workflows/cmake-multi-platform.yml:29`) — installs Qt 6.10.2 (`.github/workflows/cmake-multi-platform.yml:45`), caches the FetchContent dependencies via `ccache`, builds the entire tree including the GUI and visualizer (`.github/workflows/cmake-multi-platform.yml:68`), and runs `ctest -R 'qtrocket_*` (`.github/workflows/cmake-multi-platform.yml:74`). A fifth job boots a FreeBSD 15.0 QEMU virtual machine on the Ubuntu runner (`.github/workflows/cmake-multi-platform.yml:94`), installs Qt from `pkg` (`.github/workflows/cmake-multi-platform.yml:102`), and runs the same preset and filter. Exercising GCC, Clang, AppleClang, and MSVC under `-Werror//WX` on every merge request is real rigor for a project of this size, and the FreeBSD job goes beyond what most C++ projects of any size attempt.

Two blemishes. First, the `ctest` regex matches the `heavy` entry, so all five jobs — the QEMU VM included — fly every motor of every impulse-class ladder on every pull request: exactly the cost the `-LE heavy` fast loop exists to avoid. Second, the test step's comment still names only four suites, “(model, sim, integration, propagator)” (`.github/workflows/cmake-multi-platform.yml:72`), predating the `cli` and `design-matrix` binaries. Beyond running tests, CI publishes nothing: no coverage upload, no sanitizer or lint job, no artifacts, no releases.

9.5 The utils layer

The bottom layer is deliberately small — four components, Qt-free — and its quality is more mixed than the layers above it.

Logger A Meyers-singleton logger (`utils/Logger.cpp:19`) with five cumulative levels, an intentional-fallthrough switch under a mutex (`utils/Logger.cpp:35`), and dual sinks (`stdout` plus `log.txt`). Three defects: `setLogLevel` writes the level without taking the mutex `log()` holds while reading it (`utils/Logger.cpp:77`) — benign today, since every call site is startup or test setup, but a latent data race; `log.txt` opens relative to the process working directory (`utils/Logger.cpp:25`), so the file lands wherever the app was launched from; and entries carry level tags but no timestamps.

Bin A lower-bound interval map (`utils/Bin.h:15`) whose sole consumers are the four static US Standard Atmosphere layer tables (`sim/USStandardAtmosphere.h:28`, initialized at `sim/USStandardAtmosphere.cpp:78`). It throws `std::out_of_range` below the lowest bin, but both `operator[]` and `getBinBase` dereference `bins.begin()` before any empty check (`utils/Bin.cpp:50`) — undefined behavior on an empty `Bin` — and `insert` re-sorts the entire vector on every call (`utils/Bin.cpp:44`). Both limitations are acknowledged in-tree (`utils/Bin.h:17`) and unreachable today, the only consumers being statically populated; neither is tested.

math headers Global-namespaces Eigen aliases (`utils/math/MathTypes.h:16`), the physical-constants header whose `earthGM` feeds the spherical gravity model (`utils/math/Constants.h:26`), and a ULP-scaled `floatingPointEqual` (`utils/math/UtilityMathFunctions.h:21`). Dead weight: a 47-line commented-out block in `utils/math/MathTypes.h` (`utils/math/MathTypes.h:24`), and `earthGM_km` (`utils/math/Constants.h:27`), consumerless since the gravity path went pure SI.

CurlConnection An RAII libcurl GET wrapper: copying deleted because it would double-free the handle (`utils/CurlConnection.h:23`), reference-counted global init per instance (`utils/CurlConnection.cpp:29`), and hard 10 s connect / 30 s total timeouts with `NOSIGNAL`, justified in-tree by the synchronous GUI-thread call path (`utils/CurlConnection.cpp:54`). Transport errors and non-2xx statuses map to a logged error and an empty return (`utils/CurlConnection.cpp:68`); TLS verification is never set explicitly, relying on libcurl’s default-on peer and host checks.

9.6 Untested surfaces and dormant machinery

The coverage map has three empty regions. `utils/` has no test subdirectory at all (`utils/CMakeLists.txt:1`): `Logger`’s level semantics, `Bin`’s empty-container edge, and `floatingPointEqual` are directly untested, `Bin` being exercised only through `sim`’s atmosphere tests (`sim/tests/USStandardAtmosphereTests.cpp:5`). `CurlConnection` is never executed by any test — the network is stubbed one layer up, at the `ThrustCurveAPI` seam (Section 4) — so the one component that talks to the outside world is validated only in production. And the entire Qt surface, `gui/` and `visualizer/`, has zero automated tests: CI compiles both on every platform (`.github/workflows/cmake-multi-platform.yml:68`) but runs only the ctest suites (`.github/workflows/cmake-multi-platform.yml:74`), so the GUI is built five ways and exercised zero (Section 7). There is likewise no fuzzing of the XML, JSON, or RASP loaders that parse untrusted files.

Distribution is where the unconsumed-machinery motif is starkest. The sole install rule covers the GUI target only (`CMakeLists.txt:308`), referencing a `CMAKE_INSTALL_LIBDIR` that is never defined because `GNUInstallDirs` is never included; `qtrocket-cli`, `qtrocket-visualizer`, and the bundled `data/` motor files are not installed, there is no CPack, no desktop file, no installer of any kind, and the macOS bundle identifier is the Qt-template placeholder `my.example.com` (`CMakeLists.txt:301`). The project version, 0.1, exists only in `project()` (`CMakeLists.txt:3`) and the bundle metadata — the About dialog does not surface it and no `-version` flag or `-version` flag surfaces it. Internationalization is vestigial in the same way: `qt_create_translation` generates `QM_FILES` from the template `qtrocket_en_US.ts` (`CMakeLists.txt:201`, `CMakeLists.txt:145`), but nothing references `QM_FILES`, so no translation is ever built or shipped — `LinguistTools` is a required find-package component (`CMakeLists.txt:139`) in service of an inert call. Where OpenRocket ships platform installers, a bundled runtime, and roughly fifteen languages, QtRocket currently ships nothing at all; the comparison is drawn in Section 10, and the consolidated remediation list in Section 11.

10 Functional comparison with OpenRocket

OpenRocket is this review’s comparison baseline, and it is worth being precise about what that means: QtRocket is an independent project with similar goals, not a clone and not an implementation of OpenRocket’s specification. OpenRocket is a GPLv3 Java/Swing application, mature over nearly two decades since it originated in Sampo Niskanen’s 2009 master’s thesis, whose extended version remains the published technical documentation of its methods¹. Its current release (24.12) offers quaternion six-degree-of-freedom simulation, extended Barrowman aerodynamics, event-driven recovery and staging, and a broad component catalog — a surface QtRocket does not attempt to match today. The comparison therefore measures distance, not conformance: QtRocket currently occupies roughly the motor-database-plus-3-DOF-trajectory slice of OpenRocket’s feature space, and within that slice it is genuinely finished and unusually well tested (Sections 3 to 5).

¹<https://openrocket.info/>; technical documentation at <https://openrocket.sourceforge.net/techdoc.pdf>, including a validation chapter comparing simulations against flight and wind-tunnel data.

10.1 Four philosophical differences

The gap pattern is explained less by effort than by philosophy. First, developmental order: QtRocket builds and tests seams before consuming them. The Barrowman CP/ C_{N_q} composition (`model/parts/Part.cpp:231`), the quaternion slots in `StateData` (`sim/StateData.h:46`), and the `getTorques()` interface (`model/RocketModel.cpp:94`) all exist, are unit-tested, and are consumed by nothing — the test suite says so itself (“nothing in production consumes this in P2”, `model/tests/AeroTests.cpp:18`). OpenRocket’s equivalent machinery was built feature-first and validated against flight data afterward.

Seams before consumers

Most of the hard structural machinery for QtRocket’s next tier — composite inertia, Barrowman composition, placement resolution, adaptive integration, an event-ready termination taxonomy — is already built and tested, but staged behind seams with zero production consumers. The distance to OpenRocket is overwhelmingly a consumption gap, not a construction gap.

Second, failure posture: QtRocket is fail-closed where OpenRocket is warn-and-continue. A self-intersecting design hard-fails the mass computation with a located diagnostic (`model/parts/Part.cpp:184`) where OpenRocket warns and simulates anyway; legacy design files are rejected rather than migrated (`model/DesignSerializer.cpp:194`); the RASP parser rejects malformed motor files with typed, line-numbered errors (`model/RASPLoader.cpp:253`). This costs user forgiveness but guarantees that a produced number came from a coherent model.

Third, the placement model: OpenRocket positions components by offsets under its `AxialMethod` scheme, while QtRocket records placement as intent — a `StationLink` names fractional stations on both ends of a seam plus a seat kind selecting the radial compatibility rule (`model/parts/Placement.h:49`). Arguably the richer model, though at the pinned commit only the coaxial `Abut` verb is reachable from any front end (`cli/Repl.cpp:872`) — another built-but-unconsumed seam ([Section 3](#)).

Fourth, where the engineering effort has gone. QtRocket’s infrastructure is strong for its size: a five-platform CI matrix with `-Werror` tree-wide (`.github/workflows/cmake-multi-platform.yml:28`), `llvm-cov` coverage in the build, and 239 tests including a placement-invariance gate and 1/4A–M motor-ladder sweeps ([Section 9](#)). And three capabilities are genuine advantages rather than subsets: live `thrustcurve.org` search (`model/ThrustCurveClient.h:68`) where OpenRocket bundles periodic snapshots; a true embedded-error adaptive RK45 integrator (`sim/RK45Solver.h:75`) where OpenRocket adapts its RK4 step heuristically with no embedded error pair; and a fully scriptable headless REPL ([Section 7](#)) where OpenRocket has no interactive headless story at all.

10.2 Feature matrix

[Table 7](#) compares sixteen feature areas. OpenRocket claims are condensed from the project’s documentation and release notes²; QtRocket claims carry citations to the pinned commit. Markers: **Present** implemented and consumed; **Partial** built but incomplete or unconsumed; **Absent** not present; **WIP** under development on an unmerged branch.

²Primarily <https://openrocket.readthedocs.io/en/latest/introduction/features.html> and the release notes in the `openrocket/openrocket` repository.

Table 7. Feature-area comparison: OpenRocket 24.12 versus QtRocket at commit 254cd41.

Area	OpenRocket	QtRocket
Rocket components (part catalog)	Roughly twenty types: six nose-cone shape families, transitions, inner tubes, couplers, centering rings, lugs, rail buttons, four fin classes, recovery devices, mass objects, pods, boosters; per-component mass/CG/ C_d overrides and a commercial preset database.	Partial Five types (<code>model/parts/PartFactory.cpp:32</code>); geometry immutable after construction, so live dimension editing has no mutation path. Distinctive: intent-based <code>StationLink</code> placement (<code>model/parts/Placement.h:49</code>) whose overlap sweep hard-fails self-intersecting designs (<code>model/parts/Part.cpp:184</code>) — stronger than warn-and-simulate.
Mass / inertia modeling	Per-component mass, CG, and two moments of inertia from geometry and density, parallel-axis combined; overrides rescale or shift inertias; motor mass varies with burn time.	Present Lazy two-pass time-aware composition with parallel-axis shifts (<code>model/parts/Part.cpp:175</code>); mass-delta cache freezes bit-stably after burnout (<code>model/parts/Part.cpp:130</code>); closed-form tensors pinned by numeric oracles (<code>model/tests/InertiaTensorsTests.cpp:186</code>). The RIR^T child-tensor rotation is omitted pending 6-DOF (<code>model/parts/Part.cpp:217</code>).
Aerodynamics: CP / stability	Extended Barrowman with body lift, pitch damping, roll dynamics; CP/ $C_{N\alpha}$ evaluated per step at flight angle of attack and Mach; live CP/CG markers with caliber readout.	Partial Per-part Barrowman math exists and is unit-tested, with composite CP on a shared tip datum (<code>sim/Aero.h:24</code> , <code>model/parts/Part.cpp:231</code>) — but zero production consumers, and no surface shows CP, CG, or margin. Latent defect: mirrored <code>FinSet</code> chordwise sign convention (Section 6).
Aerodynamics: drag decomposition	Component build-up: roughness-aware skin friction, body/fin pressure drag, base drag, parasitic drag, transonic/supersonic corrections per nose shape.	Absent One user-set scalar C_d (default 1.0) times one reference area (<code>model/RocketModel.cpp:81</code>); every part's drag contribution returns zero (<code>model/parts/BodyTube.cpp:41</code>); no Mach or Reynolds number exists. <code>deriveReferenceAreaFromGeometry()</code> is correct but uncalled (<code>model/parts/Part.cpp:251</code>), so all 25 bundled designs fly on a stale or default 38 mm disc.
Degrees of freedom / flight dynamics	Quaternion 6-DOF (degrading deliberately to 3-DOF under recovery); RK4 with heuristic step reduction; validated against flight data (~10% altitude accuracy).	Partial 3-DOF point mass, heavily tested end-to-end. Thrust hardwired along world +z (<code>model/RocketModel.cpp:70</code>); <code>getTorques()</code> returns zero with no caller (<code>model/RocketModel.cpp:94</code>); orientation integrator commented out (<code>sim/Propagator.h:103</code>). Ahead: embedded-error adaptive RKF45 (<code>sim/RK45Solver.h:75</code>); typed termination taxonomy (<code>sim/Propagator.h:35</code>).
Atmosphere / gravity / wind	Eight-layer ISA with site overrides; pink-noise turbulence and multi-level winds with CSV import; flat/spherical/WGS84 earth with Coriolis; launch rod/rail.	Partial Three atmospheres (US Standard 1976 validated to 0.1–0.5% up to 80 km) and true inverse-square spherical gravity (<code>sim/SphericalGravityModel.cpp:28</code>). Wind is dead code: returns (0, 0, 0), zero call sites (<code>sim/WindModel.cpp:16</code>); no rod/rail (pad-hold clamp only, <code>sim/Propagator.cpp:105</code>), no launch site, no Coriolis.
Recovery	Parachute/streamer deployment on launch/ejection/apogee/altitude/separation events with delays; dual deployment; empirical streamer C_d ; post-deploy descent; zipper and ground-hit warnings.	Absent Nothing changes at apogee; flight ends ballistically at the $z < 0$ -descending terminate (<code>model/RocketModel.cpp:62</code>). Ejection delays are parsed and persisted but consumed by no flight logic; apogee exists only as a post-hoc statistic.

continued on next page

Table 7 (continued)

Area	OpenRocket	QtRocket
Staging / clusters / flight configurations	Unlimited serial stages plus strap-on boosters; separation and ignition event options with delays; motor clusters; named flight configurations.	Absent Single motor, single stage: first-match DFS (<code>model/RocketModel.cpp:13</code>); <code>Stage.h</code> is a commented-out “Not yet” include (<code>model/RocketModel.h:22</code>). Ignition timing is half-plumbed (<code>ThrustCurve::setIgnitionTime</code> has zero call sites); the motor sits at the airframe CM with a hardcoded zero offset (<code>model/RocketModel.cpp:124</code>).
Motor data & selection	Bundled complete thrustcurve.org corpus (1000+ motors) offline; RASP <code>.eng</code> and RockSim <code>.rse</code> user files; rich chooser with search, filters, curve plots, delay presets.	Partial Three ingest paths behind one facade: RSE (<code>model/RSEDatabaseLoader.cpp:20</code>), strict RASP (<code>model/RASPLoader.cpp:253</code>), live thrustcurve.org search behind a fake-injectable seam (<code>model/ThrustCurveClient.h:68</code>) — live search the baseline lacks. Ships with an empty database; per-sample mass/CG data discarded (<code>model/RSEDatabaseLoader.cpp:89</code>); common-name keying silently replaces on collision.
Design UI	Component tree with drag-and-drop and undo; per-component dialogs; live 2D schematics with CG/CP markers; real-time background simulation while editing.	WIP At the pinned commit, a motor-and-launch front end only: design widgets are constructor stubs (<code>gui/RocketTreeView.cpp:1</code>), File actions ship disabled (<code>gui/MainWindow.ui:116</code>), authoring is CLI-only (<code>cli/Repl.cpp:784</code>). The unmerged <code>RocketTreeViewImplementation</code> branch adds a read-only tree and <code>.qrd</code> open/save; no add/edit UI, no property panel, no schematic.
3D visualization	Embedded 3D finished/unfinished views, per-component appearance, textures and decals, PhotoStudio photorealistic rendering.	Present Standalone <code>qtrocket-visualizer</code> meshes the tree (<code>visualizer/RocketMesh.cpp:90</code>) and positions meshes with the simulator’s own placement resolver (<code>visualizer/RocketMesh.cpp:433</code>), so rendered and simulated geometry cannot diverge; overlap offenders render in error red (<code>visualizer/RocketGLWidget.cpp:617</code>). Not embedded in the GUI; motors render no geometry; no tests.
Plotting / analysis	50+ plottable variables on configurable axes with event overlays; custom expressions; component analysis; CSV export with selectable columns.	Partial Three fixed plots (<code>gui/AnalysisWindow.cpp:36</code>); CLI launch summary plus a fixed-schema CSV auto-written to a hardcoded <code>./qtrocket_run.csv</code> (<code>cli/Repl.cpp:104</code>). No configurable plots, no event annotations, no stability outputs.
File formats & interop	<code>.ork</code> zipped XML with loaders back to v1.0; RockSim <code>.rkt</code> and RASAero <code>.CDX1</code> import/export; OBJ, SVG, CSV, PDF export.	Partial Versioned <code>.qrd</code> XML round-trips the tree with placement intent, re-resolved on load (<code>model/DesignSerializer.cpp:93</code>); fail-safe atomic loading (<code>model/DesignSerializer.cpp:255</code>). Legacy 0.1 files rejected with no migration path (<code>model/DesignSerializer.cpp:194</code>); mass overrides lost on save; no foreign import or export of any kind.
Simulation extensibility / scripting	Three simulation-listener interfaces that can mutate state per step; shipped extensions; JavaScript scripting; runtime plugin JARs.	Partial No listener, plugin, or scripting mechanism. Partial substitute the baseline lacks: a 33-command scriptable CLI over a machine-parseable OK/ERR protocol (<code>cli/Repl.cpp:232</code>), weakened as automation by an always-zero exit code.

continued on next page

Table 7 (continued)

Area	OpenRocket	QtRocket
Optimization / dispersion	Rocket Optimizer: multi-parameter optimization toward altitude/velocity/stability goals. Monte Carlo dispersion explicitly absent, on its own roadmap.	Absent Neither optimizer nor dispersion exists (greps return zero hits); since the baseline also lacks Monte Carlo, the gap is the optimizer only. The deterministic core and scriptable CLI make external sweeps feasible today, but nothing in-product supports them.
Docs / i18n / platform / distribution	GPLv3 Java with JRE-bundled installers for Windows/macOS/Linux, Snap and Chocolatey packages; 15-language localization; published technical documentation with a validation chapter.	Partial Strong engineering infrastructure: five-platform CI with <code>-Werror</code> (<code>.github/workflows/cmake-multi-platform.yml:28</code>), <code>llvm-cov</code> coverage, 239 tests. Distribution essentially absent: one install rule (<code>CMakeLists.txt:308</code>), no CPack or installers; i18n is dead scaffolding (<code>CMakeLists.txt:201</code>), English only.

10.3 Deficiencies, ranked by flight-realism impact

Ranked by how much each blocks simulating a realistic flight, the gaps order as follows; the consolidated register with severities is [Section 11](#).

Rank 1: flight-dynamics fidelity. No rotational dynamics and no aerodynamics reach the force path: thrust is hardwired along world +z (`model/RocketModel.cpp:70`), `getTorques()` has no caller, the orientation integrator is commented out, and `StateData`'s quaternions are zero-initialized to a non-rotation (`sim/StateData.h:46`). The fully built Barrowman layer never reaches `getForces`, so there is no angle of attack, restoring moment, weathercocking, or gravity turn — an angled launch tilts only the initial velocity while thrust stays permanently vertical. OpenRocket integrates quaternion 6-DOF with angle-of-attack-dependent Barrowman forces every step. Two latent defects must be fixed before the seam is consumed: the `FinSet` sign mirror ([Section 6](#)) and the omitted $R I R^T$ rotation ([Section 3](#)). The force path is developed in [Sections 5](#) and [6](#).

Rank 2: recovery and flight events. Every flight ends ballistically (`model/RocketModel.cpp:62`); nothing changes at apogee. Ejection delays are ingested and persisted but drive no flight logic ([Section 4](#)). OpenRocket's event architecture — ignition, burnout, ejection, rod-clear, apogee, deployment, touchdown — drives staged recovery with per-device triggers and descent under parachute drag; QtRocket reports only a termination reason ([Section 5](#)). For anyone who cares about descent rate or landing distance, this is the most visible physics gap.

Rank 3: drag fidelity. A single user-guessed C_d times a single reference area (`model/RocketModel.cpp:81`) is a hand calculation, not a drag model; no Mach number exists, so large-motor flights sail through Mach 1 on subsonic-constant drag. The wiring bug compounds it: `deriveReferenceAreaFromGeometry()` is correct but uncalled (`model/parts/Part.cpp:251`), so drag does not even scale with airframe class ([Section 6](#)).

Rank 4: wind and launch environment. `WindModel` returns (0, 0, 0) with zero call sites (`sim/WindModel.cpp:16`), so airspeed always equals ground velocity; there is no launch rod or rail (`sim/Propagator.cpp:105`), no launch site, and no Coriolis — against pink-noise turbulence, multi-level winds, rod-departure warnings, and WGS84 geodetics in the baseline ([Section 5](#)).

Rank 5: propulsion breadth. One motor, one stage (`model/RocketModel.cpp:13`); no clusters, configurations, or ignition events; the motor pinned at the airframe CM (`model/RocketModel.cpp:124`) skews CG travel over the burn; and the RSE loader discards the manufacturer-measured mass/CG samples OpenRocket honors (`model/RSEDatabaseLoader.cpp:89`). QtRocket also ships with an empty

motor database versus OpenRocket’s bundled thousand-plus-motor corpus³ (Section 4).

Rank 6: component catalog. Five part types (`model/parts/PartFactory.cpp:32`) against roughly twenty: many realistic airframes cannot be described yet, independent of any physics limitation, and immutable part geometry leaves live dimension editing — table stakes in a design tool — with no mutation path (Section 3).

Rank 7: the GUI design experience, the widest user-facing gap. At the pinned commit the design widgets are stubs (`gui/RocketTreeView.cpp:1`) and authoring is CLI-only; the unmerged `RocketTreeViewImplementation` branch adds a read-only tree and `.qrd` open/save but no editor, and no CP/CG or stability readout exists on any surface. Simulation and network fetches run synchronously on the GUI thread, and `QApplication` runs on a spawned worker thread contrary to Qt’s expectations (`gui/GuiRunner.cpp:59`, fatal on macOS) (Section 7).

Ranks 8–10, briefly. Analysis: three fixed plots and a fixed-schema CSV versus 50+ configurable variables with custom expressions (Section 7). Interop: no `.ork` or `.rkt` import, no plugin mechanism, no optimizer — Monte Carlo is absent in both tools (Section 8). Distribution: essentially no packaging, dead i18n scaffolding, versus fifteen languages and JRE-bundled installers (Section 9).

10.4 Where QtRocket is ahead

Fairness cuts both ways. Live `thrustcurve.org` search behind a fake-injectable seam (`model/ThrustCurveClient.h:68`) has no OpenRocket equivalent; nor does the embedded-error adaptive RKF45 (`sim/RK45Solver.h:75`); nor the hang-proof loop with its typed termination taxonomy (`sim/Propagator.h:35`); nor the scriptable headless REPL (`cli/Repl.cpp:232`). The visualizer positions meshes with the exact placement resolver the physics uses (`visualizer/RocketMesh.cpp:433`), a single-source-of-truth property OpenRocket’s separate rendering path does not enjoy; gravity is available as true inverse-square spherical (`sim/SphericalGravityModel.cpp:28`); and the fail-closed overlap diagnostics (`model/parts/Part.cpp:184`) impose a correctness discipline the baseline’s warn-and-continue model does not.

The fair summary: OpenRocket is a mature product with a broad surface and a published validation record; QtRocket is a rigorously engineered core whose hard structural machinery for the next tier is already built, tested, and staged behind well-signposted seams. The roadmap-critical work (Section 11) is overwhelmingly consumption rather than construction: wire the aero profile and torques into a 6-DOF force path, add an event system with recovery, connect the geometry-derived reference area, plumb wind, and give the GUI the design editor the CLI already proves the core can support.

11 Incomplete areas & future work

Every body section has reported its gaps in place; this section collects them into one ranked inventory, records the work in flight and the branches parked behind it, separates what is *missing* from what was *deliberately deferred*, and closes with a suggested order of work — which investments unlock which others.

11.1 Incompleteness as a discipline

One finding from the incompleteness sweep deserves top billing: QtRocket’s incompleteness is unusually well-signposted. Almost every stub names the work that will consume it, and the roadmap in `TODO.md` is accurate in both directions — its completed-work claims verify against the code, and its

³Maintained separately at <https://github.com/openrocket/motor-database> and re-imported each release.

open items describe what is genuinely absent. The clearest example is the Barrowman test suite, which says outright that it is pinning scaffolding for a consumer that does not exist yet (`model/tests/AeroTests.cpp:18`):

```
// Exercises the Barrowman composition seam: per-part getAero folded by Part::getCompositeAero into a
// whole-rocket AeroProfile, with x_cp shared on the root-CM datum. NOTE: nothing in production
// consumes this in P2 -- these tests pin the seam for P5.
```

The same discipline shows in the simulation core, where the future orientation integrator sits as a commented-out `std::unique_ptr<sim::RK4Solver<Quaternion>>` member directly beside the live linear one, its interface already worked out in the accompanying comment: it will reuse the `DESolver` abstraction with a time-keyed callback so `getTorques()` can be sampled at each stage's node time exactly as `getForces()` is today (`sim/Propagator.h:100`).

Tested is not shipped

The signposting cuts both ways. A contributor can locate the frontier by reading, because the seams say what they are waiting for; but a reader skimming the tests could mistake the project for further along than it is, because several finished artifacts have zero consumers — the Barrowman layer (unit-tested, uncalled; `model/tests/AeroTests.cpp:18`), the wind model (compiled, orphaned; `sim/WindModel.cpp:16`), the torque interface (`model/RocketModel.cpp:94`), the quaternion state slots (`sim/StateData.h:46`), and the ejection-delay data every motor loader ingests (`model/RSEDatabaseLoader.cpp:59`). The roadmap below is therefore mostly a plan for *consumption*, not construction.

11.2 Consolidated inventory

Three tiers, ranked by user impact: what blocks a realistic flight, what blocks authoring and everyday use, and what merely misleads a maintainer. Tier 1 collects the gaps that [Section 10](#) ranks first by flight-realism impact — the machinery a realistic powered flight cannot do without. Each entry is a summary with citations; the owning body section carries the analysis.

Tier 1 — blocking realistic flight

Recovery phase and flight events — Absent. The only flight-phase logic in the codebase is the terminate predicate “descending below the launch site” (`model/RocketModel.cpp:65`); nothing changes at apogee, and no parachute or deployment identifier exists in production source. The data for ejection-triggered deployment is already ingested — delays are parsed from RSE files (`model/RSEDatabaseLoader.cpp:59`) and stored per motor (`model/MotorModel.h:303`) — but consumed by no flight logic; the thrustcurve.org client leaves them a TODO (`model/ThrustCurveClient.cpp:198`). The typed termination taxonomy (`sim/Propagator.h:35`) is the natural seed of an event vocabulary; see [Section 5](#) and [Section 6](#).

Rotational dynamics — Absent (dormant seams throughout). `getTorques()` returns zeros and has no production caller (`model/RocketModel.cpp:94`); thrust is hardwired along world +z regardless of attitude (`model/RocketModel.cpp:71`); the orientation integrator is the commented-out member shown above (`sim/Propagator.h:103`); and `StateData`'s quaternions are zero-initialized to a non-rotation, with a “(vector, scalar)” annotation contradicting Eigen's (w, x, y, z) constructor order (`sim/StateData.h:46`). An angled launch tilts only the initial velocity. [Section 5](#), [Section 6](#).

Barrowman seam unconsumed — Partial (built, tested, uncalled). Per-part `getAero` and the composite CP-on-the-CG-datum profile are complete and unit-tested, yet the only caller of `getCompositeAero` outside the model tests is itself a test (`tests/PlacementInvarianceTests.cpp:106`). Flight drag remains one hand-entered line (`model/RocketModel.cpp:88`), and `Propagatable::aeroData` is a default-constructed member nothing reads (`model/Propagatable.h:61`). Two latent defects gate consumption: the `FinSet` chordwise sign mirror and the omitted child-tensor rotation ([Section 6](#), [Section 3](#)).

Wind — Absent (dead code). `WindModel` is compiled into the sim library (`sim/CMakeLists.txt:24`) but orphaned: it returns (0,0,0) with all parameters ignored (`sim/WindModel.cpp:16`), `Environment` has no wind member (`sim/Environment.h:117`), and nothing instantiates it. Drag uses raw velocity, not relative airspeed (`model/RocketModel.cpp:88`); [Section 5](#).

Per-part drag build-up and reference area — Partial. Every concrete part’s drag contribution is zero, so the per-part C_d seam carries no signal ([Section 6](#)); `deriveReferenceAreaFromGeometry()` correctly implements the max-frontal-disc convention but has no production caller (`model/parts/Part.cpp:251`), so drag does not even scale with airframe class. No Mach or Reynolds number exists: the atmosphere’s temperature and pressure are pinned by tests (`sim/tests/USStandardAtmosphereTests.cpp:41`), but `USStandardAtmosphere::getSpeedOfSound` (`sim/USStandardAtmosphere.cpp:143`) and its viscosity companion have no caller of any kind, production or test.

Tier 2 — usability and authoring

GUI design editor — Absent at HEAD, WIP on a branch. At the pinned commit, `RocketTreeView` and `RocketModelerView` are seven-line constructor-only classes (`gui/RocketTreeView.cpp:3`, `gui/RocketModelerView.cpp:3`) whose widgets are already placed in the main window; File New/Open/Save/Save As/Close ship `enabled=false` (`gui/MainWindow.ui:118`), and `MainWindow` wires only Quit, SaveMotorDatabase, and About (`gui/MainWindow.cpp:42`). Design authoring is CLI-only. [Section 7](#); the in-flight branch is covered in [Section 11.3](#).

CLI placement authoring — Partial (accepts and ignores). `parseDesignTokens` parses `x=/y=/z=` (`cli/Repl.cpp:189`) but `addpart` consumes only `z` as an abut standoff (`cli/Repl.cpp:872`), and no CLI syntax reaches the `NestInBore/OnSurface` seats or non-default stations the model API and serializer fully support (`model/parts/Placement.h:134`); seat-based designs require hand-edited `.qrd XML`. [Section 7](#).

Motor placement, single motor, single stage — Partial. `motorOffset` is fixed at zero with no setter (`model/RocketModel.h:131`), the motor’s link is built from it (`model/RocketModel.cpp:124`) and never persisted (`model/DesignSerializer.cpp:114`), so every motor sits at the airframe CM, skewing $CG(t)$. One motor is structural (`findMotorInTree` is a first-match DFS, `model/RocketModel.cpp:13`); one stage likewise — `Stage.h` survives only as a “Not yet” commented include (`model/RocketModel.h:22`), the original class removed in March 2024 (commit 46eca11) to simplify prototyping. [Section 4](#).

Launch site — Absent. The spherical geoid ignores its latitude/longitude arguments (`sim/SphericalGeoidModel.cpp:18`) and the gravity model caches ground level at a self-declared placeholder (0,0) (`sim/SphericalGravityModel.cpp:20`); no ECEF transform or launch-site input exists in any front end. [Section 5](#).

Simulation synchronous on the GUI thread — Absent (no worker). `launchRocket()` runs the whole propagation inline (`QtRocket.cpp:54`) from a button slot (`gui/CannonballTab.cpp:116`). Tolerable for a seconds-long ballistic hop; a minutes-long parachute descent — the Tier 1

recovery work — will freeze the UI, so that work forces this one. The broader GUI friction set is inventoried in [Section 7](#).

Tier 3 — hygiene and latent traps

Dead and dormant fields. `Propagatable::nextState` is referenced nowhere (`model/Propagatable.h:65`); `aeroData` is never read (`model/Propagatable.h:61`); `MotorModel::isp` is computed and dropped (`model/MotorModel.cpp:130`); `StateData`’s `dcm` and `eulerAngles` are never written (`sim/StateData.h:49`); `MotorModel.h` retains commented-out serialization includes marked unused (`model/MotorModel.h:10`). Each is a trap for a reader assuming it works.

Motor metadata losses. The `thrustcurve.org` import leaves `certOrg` and delays unfilled with explicit TODOs (`model/ThrustCurveClient.cpp:197`); `infoUrl` carries “TODO: ???” (`model/MotorModel.h:307`); metadata members are public “just for testing” (`model/MotorModel.h:282`); typo’d certification strings are locked in for round-trip fidelity (`model/MotorModel.h:144`); the RSE loader discards manufacturer-measured per-sample mass/CG data (`model/RSEDatabaseLoader.cpp:89`). [Section 4](#).

Racy controller initialization. The `QtRocket` singleton (`QtRocket.h:29`) guards construction with a mutex but tests its `initialized` flag *before* taking the lock — benign today only because every front end first-calls it from one thread; real the moment simulations move to a worker thread.

Documentation drift. Four documents describe superseded designs: the persistence spec still shows the retired 0.1 <offset> schema (`docs/P2_DESIGN_PERSISTENCE_AND_CLI_SPEC.md:160`) that the shipped serializer rejects (`model/DesignSerializer.cpp:196`); the visualizer’s frozen spec documents the `Vector3`-offset scheme (`visualizer/SPEC.md:38`) the visualizer itself no longer uses (`visualizer/RocketMesh.cpp:434`); the architecture audit’s gap register is half-stale (`docs/ARCHITECTURE_AUDIT.md:355`) with citations that no longer resolve (`sim/USStandardAtmosphere.h:17`); `TODO.md` is accurate, but legacy phase numbers in code comments resolve only through its appendix translation table.

Internationalization and packaging vestiges. `QTranslator` plumbing loads a locale-suffixed translation that only resolves to US English (help-wanted TODO, `gui/GuiRunner.cpp:34`); distribution is a single template install rule with no packaging pipeline ([Section 9](#)).

Small tracked notes. The remaining marked TODOs are honest and minor: `StateData` encapsulation (`sim/StateData.h:29`), `utils::Bin` container-ification (`utils/Bin.h:17`) and its linear-scan note (`utils/Bin.cpp:39`).

11.3 Work in flight and branch archaeology

This review is pinned to commit `254cd41`, simultaneously the tip of `development` and of the merged `PartPlacement` branch. One branch is actively ahead: `RocketTreeViewImplementation`, two commits ahead (`049bb02`, `2ddd553` “GUI save/load WIP”; +360/−15 lines over ten files). It adds a read-only `QAbstractItemModel` part tree (Name/Type/Mass), a `RocketModel::setStructureChangedCallback` hook — a plain `std::function`, keeping `model/Qt-free` — plus `Part::getParent()`, and working File Open/Save/Save As slots calling `DesignSerializer`, re-enabling exactly those three actions. It adds no property panel, no part add/remove/edit UI, no modeler rendering, and no tests; New and Close stay disabled. It is the first slice of the Tier 2 GUI editor, not the editor — and all statements about it describe that branch, not the pinned commit. The remaining remotes ([Table 8](#)) are archaeology rather than roadmap: mostly merged feature branches, plus two abandoned experiments.

Remote branch	Ahead	Disposition
RocketTreeViewImplementation	+2	In flight: GUI part tree and save/load WIP (above).
questionable_changes	+1	Abandoned; its own commit message says it “doesn’t build.”
3Dviewer_test_codex	+1	Alternate visualizer prototype; superseded by the shipped visualizer/.
3Dviewer_test	0	Merged visualizer prototype.
MotorPart	0	Merged: motor-in-the-part-tree work.
ConcretePartTypes	0	Merged: concrete part types, RASP loader, coverage tooling.
propagator	0	Merged: early propagator refactor.
development-WIP	0	Fully merged 2023-era state; last commit 2023-04-27.
master	behind	Retired old default branch.
website	+32	Jekyll project site; unrelated to the code.

Table 8. Remote branches at the review commit. “Ahead” counts commits not reachable from `development`.

11.4 Deliberately deferred, not forgotten

The Part Placement design document drew a distinction worth preserving: a deferred idea is a planned extension point, not a dead end. Four of its deferrals are inherited by this roadmap, each with its extension point already built.

The joint graph. Each child stores exactly one `StationLink` to one parent (`model/parts/Part.h:288`), so a part bound to two hosts — clusters, struts, pods — is inexpressible. The implementation plan records this as “deferred (correct long-term)” (`docs/part-placement-implementation-plan.md:1031`): the single link per child can widen into a link list or joint table without disturbing the resolver’s interface.

Off-axis seating. A radial offset $r > 0$ is used only for the seam and overlap *check* while the pose solve keeps parts coaxial, because off-axis mass without an orientation would itself be a half-correct stored placement — the error class the placement work eliminated. It arrives with 6-DOF, alongside `StationLink::childRot`, already carried as an identity quaternion (`model/parts/Placement.h:55`), deliberately unpersisted (`model/DesignSerializer.cpp:85`), and deliberately unapplied in the composite inertia walk — the RIR^T rotation is omitted to stay bit-stable, with the code prescribing the test that must accompany its introduction (`model/parts/Part.cpp:220`).

Fin sets with fewer than three fins. `FinSet` warns and applies the $N \geq 3$ axisymmetric approximation rather than the true anisotropic tensor (`model/parts/FinSet.cpp:41`) — honest under-modeling that becomes a real limit once rotational dynamics exist to feel the difference.

CLI auto-stack stations. The planned `station=` shorthand never shipped; the CLI expresses only the default abut stack (`cli/Repl.cpp:872`). Part of the same Tier 2 authoring gap as the unreachable seats, and cheap relative to the model machinery that already supports it.

These deferrals converge: the 6-DOF work consumes `childRot`, the tensor rotation, and off-axis seating together, while the joint graph waits for a design that genuinely needs multi-host attachment. Deferred is not dismissed — each names the code already shaped to receive it.

11.5 Sequencing: what unlocks what

Consumption before construction

Almost nothing in Tiers 1–2 requires new architecture. The aero profile, the torque interface, the quaternion state, the ejection delays, the termination taxonomy, the authoring API, and the serializer all exist and are tested. The highest-leverage sequencing wires existing seams to consumers in dependency order, fixing the two known latent defects before their numbers become user-visible.

1. **Consume the Barrowman seam.** Prerequisites first: correct the `FinSet` chordwise sign convention, introduce the RIR^T rotation with its pinned-identity regression test (`model/parts/Part.cpp:220`), and give `deriveReferenceAreaFromGeometry()` its missing caller (`model/parts/Part.cpp:251`). The payoff is cheap and visible: a CP/CG/static-margin readout in CLI and GUI costs no new physics, exercises `getCompositeAero` for the first time, and flushes the sign defect while it is still a display bug rather than a flight bug (Section 6).
2. **Recovery and flight events.** Independent of rotation, and the most visible fidelity gap: grow the termination taxonomy (`sim/Propagator.h:35`) into an event vocabulary (liftoff, burnout, apogee, ejection, touchdown), consume the already-ingested ejection delays (`model/MotorModel.h:303`), and switch drag configuration at deployment. This replaces the ballistic terminate at `model/RocketModel.cpp:65` with a full flight — and, because descents run minutes rather than seconds, it forces the worker-thread move off the GUI thread (`QtRocket.cpp:54`), which in turn makes the initialization race worth fixing first.
3. **Six degrees of freedom, then wind.** Reinstate the orientation integrator whose interface is already designed in place (`sim/Propagator.h:103`), fixing the quaternion zero-initialization and the (w, x, y, z) -order ambiguity first (`sim/StateData.h:46`); point thrust along the body axis instead of world $+z$ (`model/RocketModel.cpp:71`); derive restoring torques from the profile consumed in step 1. Only then is wind worth plumbing: a point mass cannot weathercock, so connecting `WindModel` (`sim/WindModel.cpp:16`) earlier would shift drag without producing any behavior wind exists to model.
4. **The GUI design editor.** Pure consumption of a proven core: the CLI already demonstrates the full build–save–reload–fly loop against the same authoring API (`cli/Repl.cpp:784`), the 25 tracked `.qrd` fixtures give the editor a ready test corpus, and the `RocketTreeViewImplementation` branch supplies the tree view and the change-notification bridge. What remains is the property panel, part add/remove/edit, the 2-D modeler — and closing the CLI’s own seat/station gap (`cli/Repl.cpp:872`), which is small and makes richer fixtures authorable for the GUI work itself.

Staging and clusters (Section 4), the joint graph, and packaging stay off this critical path deliberately: none blocks the four steps, and each gets cheaper after them — staging needs the event system, clusters need the joint graph, and the Tier 3 hygiene items are best retired by whichever step touches their files. The pattern of the sequencing is the pattern of the codebase: the hard structural machinery — composite inertia, placement resolution, Barrowman composition, adaptive integration, the termination taxonomy — is built, tested, and waiting behind seams that name their consumers. The rooms are framed and inspected; the remaining work is opening the doors between them.

A Reproducing this document (toolchain)

This whitepaper is itself a small, self-contained build: a contributor who clones the repository should be able to regenerate the PDF, with every diagram, from the sources checked into `docs/`

`ArchitectureReviewLatex/`. The layout and style contract are inherited from the Part Placement design whitepaper (`docs/PartPlacementDesignLatex/`), so the two documents read as one series; this appendix records the small ways this build differs and how the review itself was produced.

A.1 Directory layout and pipeline

The master file `ArchitectureReview.tex` carries only the document scaffolding — title block, front matter, and an ordered list of `\input` directives — while the prose lives in `sections/`, one file per section. The shared style contract is isolated in `preamble.tex`: the color palette, the fonts (XCharter text, `newtxmath`, Inconsolata mono), the callout environments, and the domain macros (`\src`, `\srcf`, `\tikzfig`, the feature-status markers) that the section files depend on. A section file is just a body, never a standalone $\text{\TeX}$ program.

Unlike the Part Placement whitepaper, this document has no PlantUML stage: every figure is hand-built TikZ under `diagrams/tikz/`, included directly by `\tikzfig` and compiled in line by the main engine. The Makefile is correspondingly simpler — no Java, no `rsvg-convert`, no fetched jar:

```
make          # latexmk -pdf -shell-escape ArchitectureReview.tex
make clean    # remove LaTeX aux files and the minted cache
make veryclean # clean, and also remove ArchitectureReview.pdf
```

The document engine is `pdflatex`, driven by `latexmk` so the multi-pass dance (cross-references, the table of contents, `cleveref`, the bookmark tree) converges automatically. The one non-default requirement is `-shell-escape`: listings are typeset by `minted`, which shells out to `pygmentize` for syntax highlighting. Prerequisites are a $\text{\TeX}$ distribution providing `latexmk`, `pdflatex`, and `minted` with its `pygmentize` helper.

A.2 How this review was produced

The review is pinned to one tree state: commit `254cd41` (“Major comment cleanup”), a clean working tree, reviewed on 2026-07-06. Every `\src` citation in this document refers to that revision; line numbers will drift as the tree moves, so a reader on a later commit should treat them as anchors, not absolutes.

The analysis behind the document was performed as a fan-out of independent subsystem reviews — part tree and placement, motors, simulation core, aerodynamics, GUI and visualizer, CLI and persistence, build/test infrastructure, and a dedicated incompleteness sweep — each grounded in a full read of its files, followed by an independent adversarial verification pass that re-checked every citation and status claim against the source. The OpenRocket comparison baseline (Section 10) was researched from primary sources (`openrocket.info`, the OpenRocket documentation and technical papers, and the `openrocket/openrocket` repository), then reconciled against the verified QtRocket findings. Documents under `docs/` were deliberately treated as claims to test, not facts to inherit: where a design document and the code disagree, this review reports the code and flags the discrepancy.

Two behaviors were additionally confirmed by execution rather than reading: the CLI command inventory of Section 7 was captured from a live `build/qtrocket-cli` session, and end-to-end flight behavior was exercised through the same REPL against a bundled design.